

How TCD-JEPA Works

A neural network that grows new predictors while it trains, by measuring the shape of its own uncertainty. This is every part of it, in order, with the reasoning behind each decision.

direncode/tcd-jepa · marketed as Crystara

54 modules · 186 tests · Apache 2.0

All code shown is read verbatim from the repository

CONTENTS

I	The idea Why predict embeddings, and why a fixed predictor is a limitation
II	The substrate The ordinary JEPA underneath, and why each piece is shaped the way it is
III	The mask is the task Why blocks, why four of them, why index tensors
IV	Splitting into three Why the architecture is three systems and not one
V	Exploring instead of descending Why a random walk, and why it is steered toward ignorance
VI	Measuring shape Why persistent homology, and why exactly three kinds of module
VII	Letting the modules back in Why the return path has three independent brakes
VIII	Three clocks Why each system runs at a different rate, and how the loop knows it is done
IX	Keeping it alive Why the training loop has four places it refuses to do the normal thing
X	The same machine on graphs Why swapping the input layer is all that graph mode required
XI	What the evidence says Where it works, where it does not, and what does not replicate
XII	Reference Commands, knobs, equations, and the one install that silently changes everything

The idea

Everything in this repository follows from one dissatisfaction: a JEPA's predictor is designed by a person, and there is no reason to believe the shape a person chose matches the shape of the problem.

Start with the thing being extended. A Joint Embedding Predictive Architecture learns by hiding part of an input, encoding what remains, and predicting what the hidden part would *encode to*. Not what it looks like — what it encodes to. There is no decoder anywhere in this codebase and no pixel ever gets reconstructed.

That choice is the foundation everything else sits on, so it is worth being precise about why it is made.

WHY PREDICT EMBEDDINGS INSTEAD OF PIXELS?

Because most of the detail in a pixel is unpredictable and worthless. Reconstruct a hidden patch of grass and the model must commit to the exact position of every blade. It cannot know them, so it spends capacity learning to hedge — producing blurry averages — and the gradient signal that shaped that capacity taught it nothing about grass.

Predicting in latent space lets the target itself decide what is worth representing. If blade positions are not encoded, they are not predicted, and no capacity is spent on them. The model is graded on the abstraction rather than on the rendering.

The cost of that choice is that the target is no longer fixed. In pixel reconstruction the answer key is the input; it cannot be gamed. In latent prediction the answer key is produced by a network that is also learning, and there is a trivial way to score perfectly: both encoders agree to output the same constant for every input. Loss zero, information zero. This is representation collapse, and preventing it drives several of the design decisions in Part II.

Now the dissatisfaction. In a standard JEPA the predictor is a transformer of a chosen depth and width. Someone picked those numbers. The architecture is fixed before training begins and never changes, no matter what the data turns out to look like.

WHY SHOULD THAT BOTHER ANYONE?

Because the predictor is where the model's theory of the data lives, and a fixed architecture asserts that theory in advance. If the data has periodic structure, a stack of attention blocks can approximate periodicity — but nothing in it is *shaped* like a cycle, and it must spend capacity discovering that shape from scratch every time.

The proposal in this repository is that the model should notice such structure in its own behaviour and build a predictor to match. Not adjust weights within a fixed shape — add new shapes.

Which raises the question the whole architecture is an answer to: **where would the model look to find out what shapes it needs?** It cannot look at the data — it has no labels, and the data is exactly what it does not yet understand. The answer this codebase gives is that it should look at its own uncertainty. Specifically: at the geometry of the regions where its predictions are least determined.

That answer decomposes into three problems, and those three problems are the three systems. Find the uncertain regions. Move through them and record the path. Measure the shape of the paths and build something matching that shape. Each system solves one, hands its product to the next, and the last hands its product back to the first.

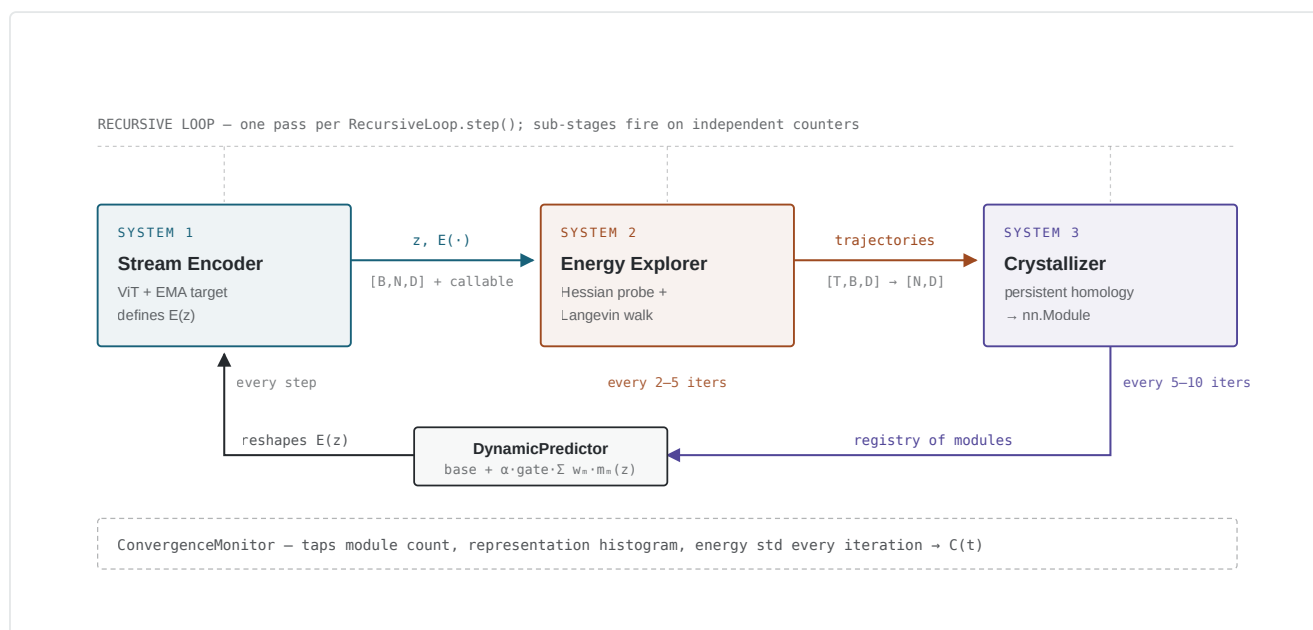


FIGURE 1 The circuit that the rest of this document explains. Each arrow carries a concrete object — latents and an energy callable, then trajectories, then `nn.Module` instances. The return path matters most: crystallized modules join the predictor, the predictor is what defines the energy surface, and so the next round of exploration happens on a landscape the previous round changed.

Read that figure once more with the cadences in mind. Exploration does not run every step and crystallization does not run every exploration. Part VIII explains why those rates differ; for now the point is that the loop is not a tight cycle but three processes on three clocks, and the slowest one is the one that changes the architecture.

The substrate

Before anything self-organizes there is an ordinary JEPA, and almost every detail of it is a defence against the collapse problem from Part I.

Four components. A **context encoder** `s_θ` that sees a few visible blocks. A **predictor** `p_φ` handed those encodings plus positional placeholders for the hidden blocks. A **target encoder** `s_ξ` that encodes the hidden blocks to produce the answer. And a loss between prediction and answer.

$$\begin{aligned}
 E(x, y) &= \| s_{\theta}(x) - s_{\xi}(y) \|^2 \\
 E_{\text{pred}}(x, y) &= \| p_{\phi}(s_{\theta}(x)) - \text{sg}[s_{\xi}(y)] \|^2 && \text{sg} \\
 \xi &\leftarrow m \cdot \xi + (1-m) \cdot \theta && m:
 \end{aligned}$$

The third line is the anti-collapse mechanism. The target encoder is not trained. It is a copy of the context encoder whose parameters are dragged along by exponential moving average, and gradients are blocked from entering it twice over — once by `torch.no_grad()` and again by an explicit `.detach()`.

WHY AN EMA COPY RATHER THAN A SECOND TRAINED ENCODER?

Collapse requires both encoders to agree to degenerate *together*. If the target is trained, that agreement is reachable by gradient descent — it is a genuine minimum and the optimizer will find it.

An EMA target cannot participate. It only ever moves toward where the context encoder already was, with a lag of roughly $1/(1-m)$ steps. At $m = 0.996$ that is a 250-step lag. The context encoder is therefore always predicting a slightly stale version of itself, and a constant output would have to be a fixed point of that lagged dynamic — reachable in principle, but the optimizer gets no gradient pointing toward it, because the target does not move in response to the context encoder's attempt to make it move.

The momentum schedule runs from 0.996 *up to* 1.0 across training. Early on the target should track a rapidly improving encoder; late on it should be nearly frozen so the objective stops moving underneath the optimizer.

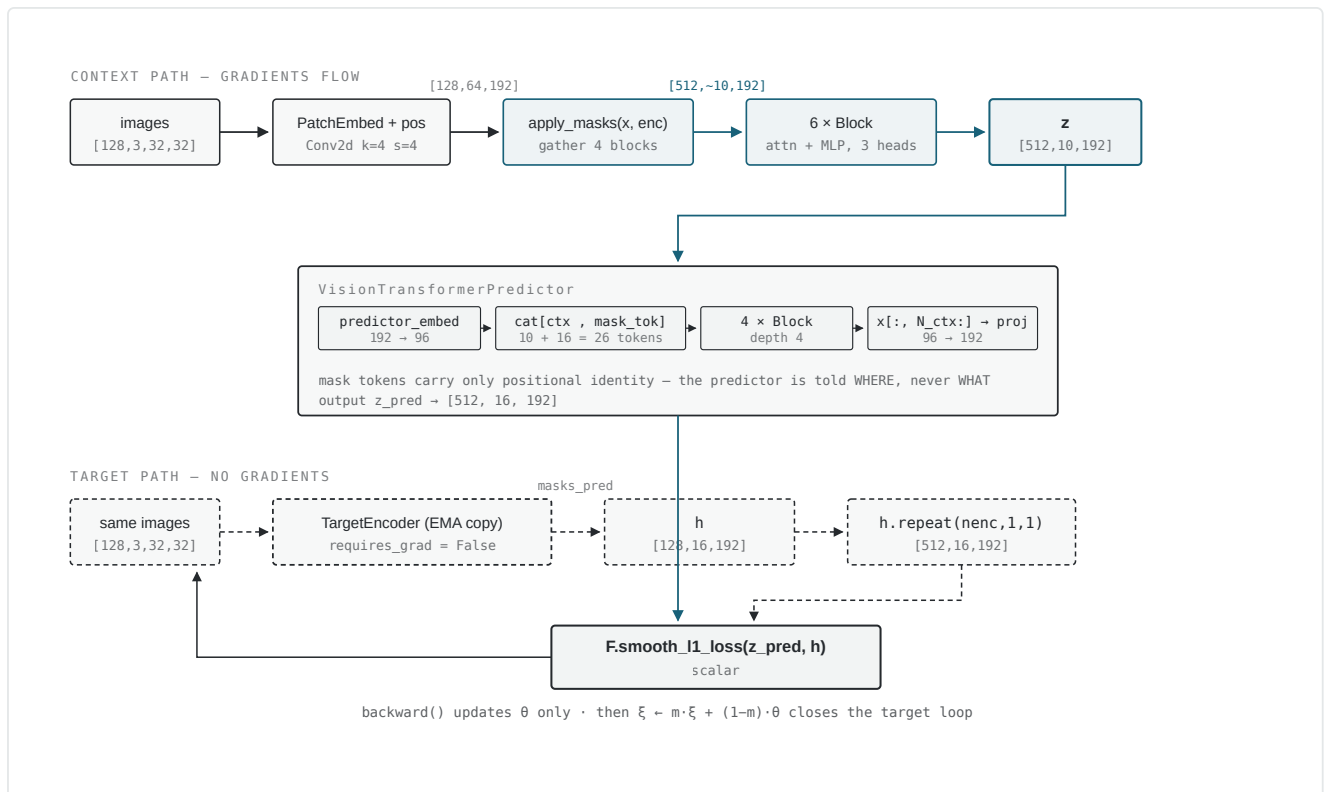


FIGURE 2 One forward pass with every shape annotated, at configs/small_scale.yaml settings: batch 128, 32×32 images, patch 4 giving 64 patches, embed_dim 192, four context masks and one target mask. Watch the batch dimension: it multiplies by four at the mask gather and stays multiplied, which is why the target is repeated four times before the loss.

Three shape facts in that figure explain most of the code you will read later.

First, masking happens *after* positional embeddings are added. The surviving tokens still know where on the image they came from. Without that, the predictor's positional mask tokens would be meaningless — it would be told to predict "position 34" without any of its inputs knowing where position 34 is relative to them.

Second, masks are index tensors, not booleans, and applying one concatenates along the batch dimension rather than zeroing entries in place.

```

tcd_jepa/utils/tensors.py lines 25-49
25 def apply_masks(x: torch.Tensor, masks: list[torch.Tensor]) -> torch.Tensor:
26     """Apply index masks to select patches from a tensor.
27
28     Args:
29         x: Tensor of shape [B, N, D] (batch, num_patches, feature_dim).
30         masks: List of tensors containing patch indices to keep.
31
32     Returns:
33         Concatenated masked tensors along the batch dimension.
34     """
35     all_x = []
36     for m in masks:
37         mask_keep = m.unsqueeze(-1).repeat(1, 1, x.size(-1))
38         all_x.append(torch.gather(x, dim=1, index=mask_keep))
39     return torch.cat(all_x, dim=0)
40
41
42 def repeat_interleave_batch(x: torch.Tensor, B: int, repeat: int) -> torch.Tensor:
43     """Repeat-interleave tensor blocks along batch dimension."""

```



```

44     N = len(x) // B
45     x = torch.cat(
46         [torch.cat([x[i * B : (i + 1) * B] for _ in range(repeat)], dim=0) for i in range(N)],
47         dim=0,
48     )
49     return x

```

`apply_masks` is the load-bearing primitive of the architecture. Everything else inherits its convention.

WHY GATHER-AND-CONCATENATE RATHER THAN A BOOLEAN MASK?

Because the blocks have different sizes and there are several of them per image. A boolean mask keeps the tensor at full length and forces the encoder to process tokens it will discard — wasted compute on 90% of the sequence.

Gathering produces a short sequence containing only real tokens. Stacking four different context blocks along the batch axis instead of the token axis then lets the encoder treat them as four independent short sequences, which is what they are — four separate views of the same image. The cost is that batch size silently multiplies by `nenc`, and every downstream shape has to account for it.

Third, the predictor is *narrower* than the encoder: 96 dimensions against 192, projecting back up at the end.

WHY BOTTLENECK THE PREDICTOR?

Because a predictor with as much capacity as the encoder can memorise instead of predict. Given enough width it can learn an idiosyncratic lookup from context patterns to target patterns without either representation becoming meaningful.

Squeezing it forces pressure back onto the encoder. If the predictor cannot represent the mapping, the only way to reduce the loss is for the encoder to produce targets that are *actually predictable* from context — which is the representation you wanted.

tcd_jepa/models/tcd_jepa_model.py

lines 45–85

```

45     def forward(

```

```

46         self,
47         images: torch.Tensor,
48         masks_enc: list[torch.Tensor],
49         masks_pred: list[torch.Tensor],
50     ) -> dict[str, torch.Tensor]:
51         """Run the full JEPA forward pass.
52
53         Args:
54             images: Input images [B, C, H, W].
55             masks_enc: List of context mask index tensors (patches to keep for encoder).
56             masks_pred: List of target mask index tensors (patches to predict).
57
58         Returns:
59             Dictionary with 'loss', 'predictions', and 'targets'.
60         """
61         # Encode context patches
62         z = self.context_encoder(images, masks=masks_enc)
63
64         # Predict target representations
65         z_pred = self.predictor(z, masks_enc, masks_pred)
66
67         # Encode targets (no gradient)
68         with torch.no_grad():
69             h = self.target_encoder(images, masks=masks_pred)
70             # Repeat targets to match predictor output shape
71             h = h.repeat(len(masks_enc), 1, 1)
72
73         # Compute loss
74         loss = jepa_loss(z_pred, h.detach())
75
76         return {
77             "loss": loss,
78             "predictions": z_pred,
79             "targets": h,
80         }
81
82     @torch.no_grad()
83     def update_target_encoder(self, momentum: float) -> None:
84         """Update target encoder via EMA."""
85         self.target_encoder.update_ema(self.context_encoder, momentum)

```

The whole JEPA objective in forty lines. Lines 68–71 run the target branch under `no_grad` ; line 74 detaches again. Redundant, deliberately.

tcd_jepa/modules/predictor.py

lines 105–161

```

105     def forward(
106         self,
107         x: torch.Tensor,
108         masks_x: list[torch.Tensor],
109         masks: list[torch.Tensor],
110     ) -> torch.Tensor:
111         """Predict target representations from context representations.
112
113         Args:
114             x: Encoded context patches [B*nenc, N_ctx, D].
115             masks_x: List of context mask index tensors.
116             masks: List of target mask index tensors (positions to predict).
117
118         Returns:
119             Predicted target representations [B*npred*nenc, N_pred, D].
120         """
121         assert masks is not None and masks_x is not None, (
122             "Cannot run predictor without mask indices"
123         )
124
125         if not isinstance(masks_x, list):
126             masks_x = [masks_x]
127         if not isinstance(masks, list):
128             masks = [masks]
129
130         B = len(x) // len(masks_x)

```

```

131
132     # Map from encoder-dim to predictor-dim
133     x = self.predictor_embed(x)
134
135     # Add positional embedding to context tokens
136     x_pos_embed = self.predictor_pos_embed.repeat(B, 1, 1)
137     x += apply_masks(x_pos_embed, masks_x)
138
139     _, N_ctxt, D = x.shape
140
141     # Prepare mask tokens with positional embeddings for target positions
142     pos_embs = self.predictor_pos_embed.repeat(B, 1, 1)
143     pos_embs = apply_masks(pos_embs, masks)
144     pos_embs = repeat_interleave_batch(pos_embs, B, repeat=len(masks_x))
145
146     pred_tokens = self.mask_token.repeat(pos_embs.size(0), pos_embs.size(1), 1)
147     pred_tokens += pos_embs
148
149     # Concatenate context and mask tokens
150     x = x.repeat(len(masks), 1, 1)
151     x = torch.cat([x, pred_tokens], dim=1)
152
153     # Forward through predictor transformer
154     for blk in self.predictor_blocks:
155         x = blk(x)
156     x = self.predictor_norm(x)
157
158     # Return only predictions for mask token positions
159     x = x[:, N_ctxt:]
160     x = self.predictor_proj(x)
161     return x

```

Lines 146–147 build the prediction slots: one learned `mask_token` broadcast to every target position, plus that position's positional embedding. Line 159 throws away the context outputs and keeps only the masked positions.

WHY ONE SHARED MASK TOKEN RATHER THAN ONE PER POSITION?

Because the token must carry no information about what it is standing in for. A single learned vector, identical everywhere, plus a positional embedding, tells the predictor exactly one thing: *where* to predict. Per-position tokens would give it a second channel it could learn to exploit, and the task would quietly become easier than intended.

The loss

tcd_jepa/training/losses.py

lines 11–34

```

11 def jepa_loss(predictions: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
12     """Standard JEPA loss: smooth L1 between predicted and target embeddings.
13
14     Matches I-JEPA's loss: F.smooth_l1_loss(z, h) where z is the prediction
15     and h is the stop-gradient target.
16
17     Args:
18         predictions: Predicted target embeddings from predictor.
19         targets: Actual target embeddings from target encoder (detached).
20
21     Returns:

```

```

22         Scalar loss value.
23         """
24         if __debug__:
25             if torch.isnan(predictions).any():
26                 logger.warning("NaN detected in predictions input to jepa_loss")
27             if torch.isnan(targets).any():
28                 logger.warning("NaN detected in targets input to jepa_loss")
29         return F.smooth_l1_loss(predictions, targets)
30
31
32 def l2_loss(predictions: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
33     """L2 loss between predictions and targets."""
34     return F.mse_loss(predictions, targets)

```

WHY SMOOTH L1 RATHER THAN PLAIN MSE?

Smooth L1 is quadratic near zero and linear beyond it. Latent targets are moving — the EMA encoder shifts every step — so occasional large residuals are normal rather than a sign of a bad prediction. Under MSE those residuals dominate the gradient and drag the model toward whichever targets happened to drift most that step. The linear tail bounds their influence.

The comparison the repository cares about is with I-JEPA, which uses the same loss; matching it keeps the baseline honest.

The target encoder's unusual safety net

tcd_jepa/models/target_encoder.py

lines 33–54

```

33 @torch.no_grad()
34 def update_ema(self, context_encoder: nn.Module, momentum: float) -> None:
35     """Update target encoder parameters via EMA.
36
37     Args:
38         context_encoder: The context encoder (or its underlying encoder).
39         momentum: EMA momentum in [0, 1]. Higher = slower update.
40         target = momentum * target + (1 - momentum) * context
41     """
42     # Handle ContextEncoder wrapper
43     if hasattr(context_encoder, "encoder"):
44         context_encoder = context_encoder.encoder
45
46     for param_q, param_k in zip(context_encoder.parameters(), self.encoder.parameters()):
47         param_k.data.mul_(momentum).add_((1.0 - momentum) * param_q.detach().data)
48
49     # Emergency NaN recovery: if EMA produced NaN, restore from context encoder
50     for param_q, param_k in zip(context_encoder.parameters(), self.encoder.parameters()):
51         if torch.isnan(param_k.data).any():
52             logger.error("NaN detected in target encoder after EMA update – restoring from context
encoder")
53             param_k.data.copy_(param_q.detach().data)
54             break

```

An EMA update, followed by a second pass that scans for NaN and restores from the context encoder if it finds any.

WHY BOTHER? NOTHING SHOULD EVER PRODUCE NAN HERE.

Because the target encoder is the one component that gradient descent cannot repair. Corrupt the context encoder and the next few steps push it back toward something sane. Corrupt the target encoder and every subsequent target is NaN, every loss is NaN, and training is over — silently, with no error, until someone reads the logs.

The asymmetry justifies the extra pass. This same reasoning is why the training loop in Part IX has a whole separate skip path that protects the EMA and nothing else.

The mask is the task

Nothing else in the config file changes what the model learns as directly. The mask defines the question; the architecture only answers it.

Masks are generated in the dataloader's collate function, not in the model. One call produces a single large target block covering 20–40% of the patch grid, and four small context blocks covering 15–20% each, sampled from the complement of the target so the two never overlap.

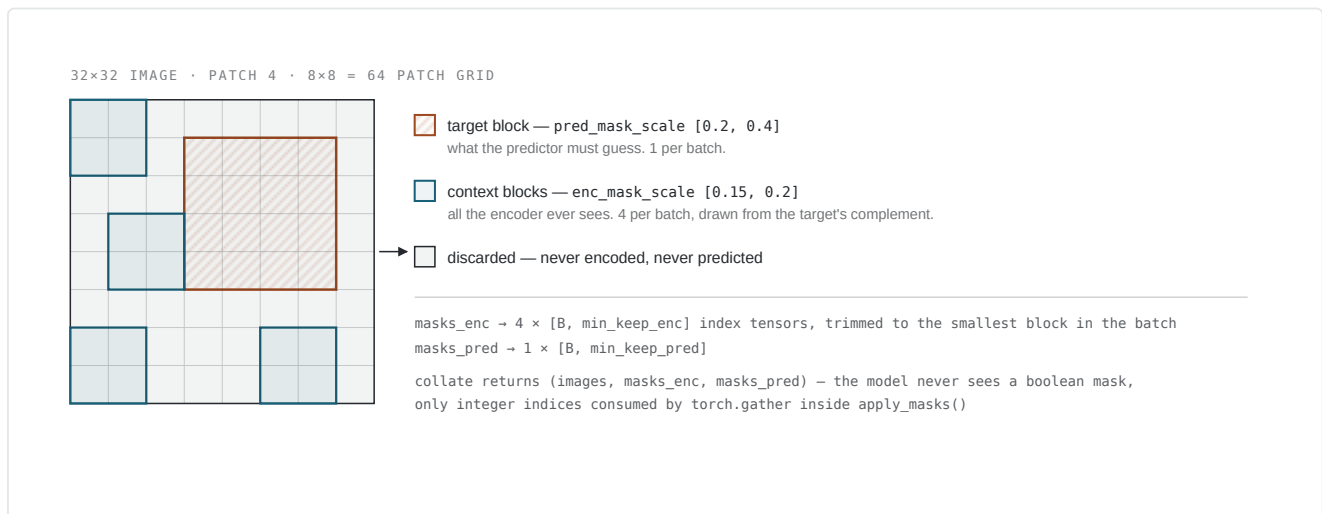


FIGURE 3 Mask layout for one batch. Blocks are resampled every batch but shared across all images in it — one geometry per step, not one per image.

WHY BLOCKS, RATHER THAN SCATTERED INDIVIDUAL PATCHES?

Because scattered patches make the task too easy in an uninteresting way. A single hidden patch surrounded by visible ones can be filled in by local interpolation — the model learns texture continuation, which is genuine but shallow.

A contiguous block removes the local context entirely. To predict its centre the model must use information from elsewhere in the image, which means it must have learned something about what the image *is*, not just how its textures flow.

WHY FOUR CONTEXT MASKS AGAINST ONE TARGET?

Two reasons, one conceptual and one about cost. Conceptually, four different partial views predicting the same target teaches the encoder that the target is inferable from many directions — a stronger constraint than a single view provides.

Practically, the target encoder pass is expensive and its result can be reused. Encoding the target once and predicting it from four contexts costs one target pass and four cheap context passes, rather than four of each. That is why the target is repeated with `h.repeat(len(masks_enc), 1, 1)` rather than recomputed.

WHY IS THE SAME MASK GEOMETRY SHARED ACROSS EVERY IMAGE IN THE BATCH?

So the tensors stack. Per-image masks would produce ragged index tensors requiring padding and a validity mask through every downstream operation. Sharing geometry within a batch and resampling between batches gets nearly the same diversity over an epoch at none of that complexity.

The visible consequence is in the collator: every mask in a batch is trimmed to the smallest one found, so a batch's blocks are equal-length even when the sampler produced slightly different sizes.

tcd_jepa/utils/masking.py

lines 119-169

```
119     def __call__(
120         self, batch: list
121     ) -> tuple[torch.Tensor, list[torch.Tensor], list[torch.Tensor]]:
122         """Create encoder and predictor masks when collating images into a batch."""
123         B = len(batch)
124         collated_batch = torch.utils.data.default_collate(batch)
125
126         seed = self.step()
127         g = torch.Generator()
128         g.manual_seed(seed)
129
130         p_size = self._sample_block_size(
131             generator=g, scale=self.pred_mask_scale, aspect_ratio_scale=self.aspect_ratio
132         )
133         e_size = self._sample_block_size(
134             generator=g, scale=self.enc_mask_scale, aspect_ratio_scale=(1.0, 1.0)
135         )
136
137         collated_masks_pred, collated_masks_enc = [], []
138         min_keep_pred = self.height * self.width
```

```

139         min_keep_enc = self.height * self.width
140
141     for _ in range(B):
142         masks_p, masks_C = [], []
143         for _ in range(self.npred):
144             mask, mask_C = self._sample_block_mask(p_size)
145             masks_p.append(mask)
146             masks_C.append(mask_C)
147             min_keep_pred = min(min_keep_pred, len(mask))
148             collated_masks_pred.append(masks_p)
149
150         acceptable_regions = None if self.allow_overlap else masks_C
151
152         masks_e = []
153         for _ in range(self.nenc):
154             mask, _ = self._sample_block_mask(e_size, acceptable_regions=acceptable_regions)
155             masks_e.append(mask)
156             min_keep_enc = min(min_keep_enc, len(mask))
157             collated_masks_enc.append(masks_e)
158
159     # Trim masks to minimum kept size and collate
160     collated_masks_pred = [
161         [cm[:min_keep_pred] for cm in cm_list] for cm_list in collated_masks_pred
162     ]
163     collated_masks_pred = torch.utils.data.default_collate(collated_masks_pred)
164
165     collated_masks_enc = [
166         [cm[:min_keep_enc] for cm in cm_list] for cm_list in collated_masks_enc
167     ]
168     collated_masks_enc = torch.utils.data.default_collate(collated_masks_enc)
169

```

Line 126 pulls a process-shared counter under a lock and seeds a fresh generator, so multiprocessing dataloader workers produce different layouts without colliding and the whole thing stays reproducible. Lines 159–168 do the trimming.

tcd_jepa/utils/masking.py

lines 80–117

```

80     def _sample_block_mask(
81         self,
82         b_size: tuple[int, int],
83         acceptable_regions: Optional[list[torch.Tensor]] = None,
84     ) -> tuple[torch.Tensor, torch.Tensor]:
85         """Sample a block mask and its complement."""
86         h, w = b_size
87
88     def constrain_mask(mask: torch.Tensor, tries: int = 0) -> None:
89         N = max(int(len(acceptable_regions) - tries), 0)
90         for k in range(N):
91             mask *= acceptable_regions[k]
92
93     tries = 0
94     timeout = og_timeout = 20
95     valid_mask = False
96     while not valid_mask:
97         top = torch.randint(0, max(self.height - h, 1), (1,))
98         left = torch.randint(0, max(self.width - w, 1), (1,))
99         mask = torch.zeros((self.height, self.width), dtype=torch.int32)
100         mask[top : top + h, left : left + w] = 1
101         if acceptable_regions is not None:
102             constrain_mask(mask, tries)
103         mask = torch.nonzero(mask.flatten())
104         valid_mask = len(mask) > self.min_keep
105         if not valid_mask:
106             timeout -= 1
107             if timeout == 0:
108                 tries += 1
109                 timeout = og_timeout
110                 logger.warning(
111                     f"Valid mask not found, decreasing acceptable-regions [{tries}]"
112                 )
113     mask = mask.squeeze()

```



```
114
115     mask_complement = torch.ones((self.height, self.width), dtype=torch.int32)
116     mask_complement[top : top + h, left : left + w] = 0
117     return mask, mask_complement
```

Rejection sampling with an escape hatch.

WHY DOES THE SAMPLER HAVE A TIMEOUT AND A RELAXATION STEP?

Because the constraint can be unsatisfiable. Asking for a context block of a given size drawn entirely from the complement of a large target block, on a small grid, sometimes has no solution. A naive rejection sampler loops forever and the training run hangs with no error.

After twenty failed attempts the sampler drops one constraint from `acceptable_regions` and tries again, logging as it goes. It degrades the mask quality rather than the run.

Splitting into three

The division into three systems is not organisational tidiness. It is what makes the same code run on images and on a four-million-edge graph without modification.

System 1 has the least interesting job and the most structurally important one. It is the adapter. It does exactly two things: it instruments the encoder so representation flow is observable, and it manufactures the energy function that System 2 will walk over.

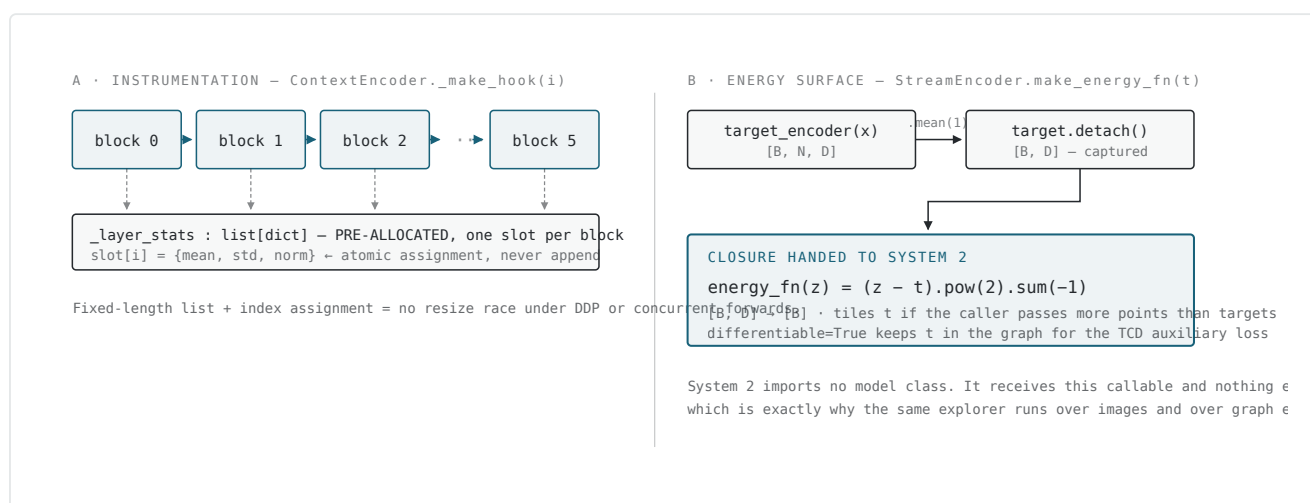


FIGURE 4 System 1's two products. Left: per-layer statistics written into pre-allocated slots. Right: the energy closure — the only interface Systems 2 and 3 have to the model.

```
tcd_jepa/core/system1_encoder.py lines 15-90

15 class StreamEncoder:
16     """System 1: provides latent representations and energy functions.
17
18     Acts as the bridge between the JEPa encoder and the exploration/
19     crystallization systems. Provides energy_fn and predictor_fn
20     callables for Systems 2 and 3.
21     """
22
23     def __init__(
24         self,
25         context_encoder: ContextEncoder,
26         target_encoder: TargetEncoder,
27     ) -> None:
28         self.context_encoder = context_encoder
29         self.target_encoder = target_encoder
30
31     def encode_context(
32         self,
33         images: torch.Tensor,
34         masks: Optional[list[torch.Tensor]] = None,
35     ) -> torch.Tensor:
36         """Encode images through context encoder."""
37         return self.context_encoder(images, masks=masks)
38
39     def encode_target(
```

```

40         self,
41         images: torch.Tensor,
42         masks: Optional[list[torch.Tensor]] = None,
43     ) -> torch.Tensor:
44         """Encode images through target encoder."""
45         return self.target_encoder(images, masks=masks)
46
47     def make_energy_fn(
48         self,
49         target_representations: torch.Tensor,
50         differentiable: bool = False,
51     ) -> callable:
52         """Create an energy function for System 2.
53
54         The energy at a point z is defined as  $E(z) = ||z - \text{target}||^2$ 
55         averaged over target patches.
56
57         Args:
58             target_representations: Target encoder outputs [B, N, D] or [B, D].
59             differentiable: If True, keeps targets in the computation graph
60                             so gradients can flow back through the energy landscape.
61
62         Returns:
63             Callable mapping [B, D] -> [B] energy.
64         """
65         if target_representations.dim() == 3:
66             target_mean = target_representations.mean(dim=1) # [B, D]
67         else:
68             target_mean = target_representations
69
70         if differentiable:
71             # Keep targets in the computation graph for gradient flow
72             target = target_mean
73         else:
74             target = target_mean.detach()
75
76         def energy_fn(z: torch.Tensor) -> torch.Tensor:
77             # Broadcast: if z has more samples than targets, tile targets
78             B_z = z.shape[0]
79             B_t = target.shape[0]
80             if B_z != B_t:
81                 t = target.repeat((B_z + B_t - 1) // B_t, 1)[:B_z]
82             else:
83                 t = target
84             return (z - t).pow(2).sum(dim=-1)
85
86         return energy_fn
87
88     def get_layer_stats(self) -> list[dict]:
89         """Get representation statistics from last forward pass."""
90         return self.context_encoder.get_layer_stats()

```

The entire adapter. Note what is *not* here: no vision, no transformers, no assumption about what produced the representations.

WHY HAND OUT A CLOSURE INSTEAD OF A REFERENCE TO THE MODEL?

This is the single decision that graph mode depends on. System 2's Langevin sampler, blank space detector and Fisher metric all accept a bare callable mapping $[B, D] \rightarrow [B]$. None of them imports a model class. None of them can tell what kind of data it is exploring.

The consequence is in Part X: swapping the vision encoder for a graph encoder required writing a new encoder, a new masking scheme and a new dataset — and changing nothing at all in the exploration or crystallization code. Had System 2 been written against a model object, that transfer would have been a rewrite.

The broadcasting rule at lines 78–83 exists because of a shape mismatch that shows up immediately in practice. System 2 subsamples up to 256 latent points from a flattened $[B \cdot N, D]$ tensor, which is almost never equal to the target batch size. Rather than constrain the caller, the closure tiles its captured target to match whatever it is handed.

tcd_jepa/models/context_encoder.py

lines 23–72

```
23 def __init__(self, encoder: VisionTransformer):
24     super().__init__()
25     self.encoder = encoder
26     # Pre-allocate fixed-length stats list – one slot per block.
27     # Hooks only do atomic assignment to a pre-existing index,
28     # avoiding append/resize races under DDP or concurrent forwards.
29     num_blocks = len(encoder.blocks)
30     self._layer_stats: list[dict[str, float]] = [{ } for _ in range(num_blocks)]
31     self._hooks: list[torch.utils.hooks.RemovableHook] = []
32     self._register_hooks()
33
34     @property
35     def embed_dim(self) -> int:
36         return self.encoder.embed_dim
37
38     @property
39     def num_heads(self) -> int:
40         return self.encoder.num_heads
41
42     @property
43     def patch_embed(self) -> nn.Module:
44         return self.encoder.patch_embed
45
46     def _register_hooks(self) -> None:
47         """Register forward hooks on each transformer block to collect statistics."""
48         for i, block in enumerate(self.encoder.blocks):
49             hook = block.register_forward_hook(self._make_hook(i))
50             self._hooks.append(hook)
51
52     def _make_hook(self, layer_idx: int):
53         """Create a hook that records statistics for a specific layer."""
54         def hook_fn(module: nn.Module, input: tuple, output: torch.Tensor) -> None:
55             with torch.no_grad():
56                 if isinstance(output, torch.Tensor):
57                     # Atomic assignment to pre-allocated slot – no append/resize.
```

```

58         self._layer_stats[layer_idx] = {
59             "mean": output.mean().item(),
60             "std": output.std().item(),
61             "norm": output.norm(dim=-1).mean().item(),
62         }
63     return hook_fn
64
65     def get_layer_stats(self) -> list[dict[str, float]]:
66         """Return statistics from the most recent forward pass."""
67         return list(self._layer_stats)
68
69     def clear_stats(self) -> None:
70         """Clear collected statistics."""
71         for i in range(len(self._layer_stats)):
72             self._layer_stats[i] = {}

```

Lines 26–30 carry the fix for a real bug.

WHY PRE-ALLOCATE THE STATISTICS LIST INSTEAD OF APPENDING TO IT?

Because appending resizes, and resizing is not atomic. Under DDP — or any situation with overlapping forward passes — two hooks appending concurrently produce a list of the wrong length, with entries attributed to the wrong layers. The failure is silent and the resulting statistics are quietly wrong.

A fixed-length list written by index cannot exhibit that. Each hook assigns to a slot that already exists. Worst case under concurrency is that a slot holds the newer of two passes, which is fine — these are diagnostics, not gradients.

Why the layering is the way it is

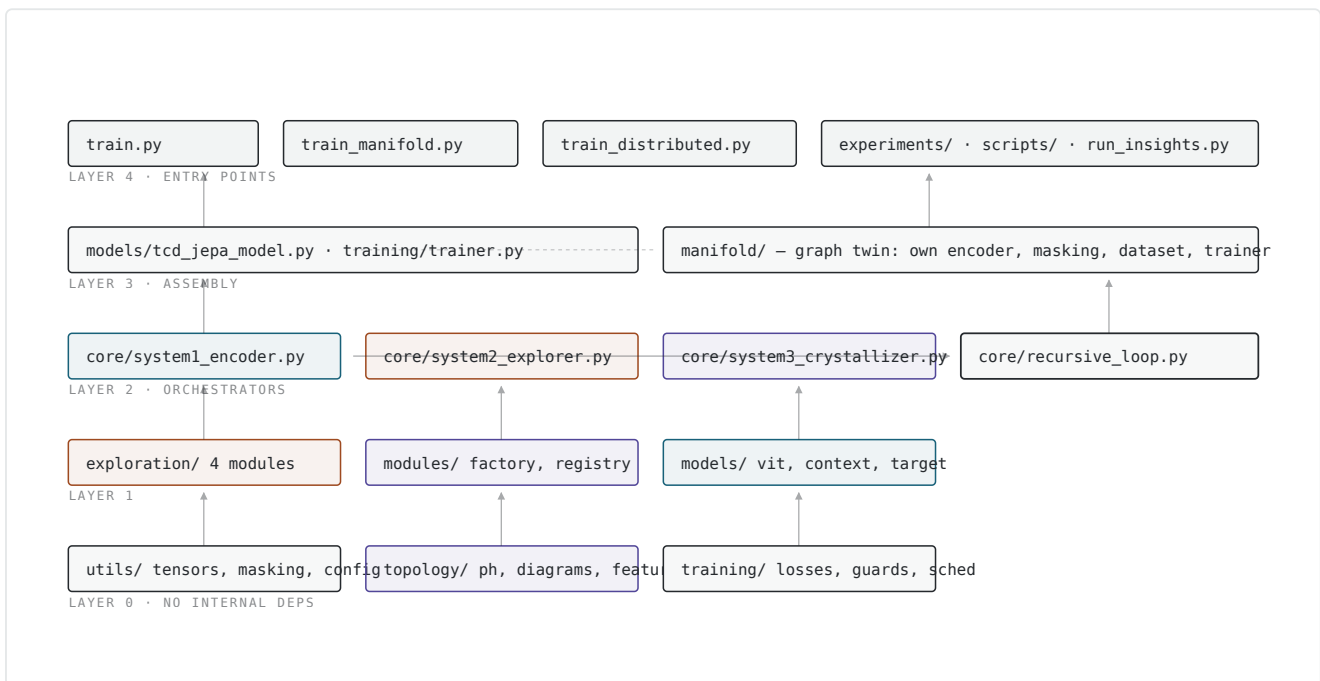


FIGURE 5 Import layers. `topology/` sits at the base and knows nothing about images, transformers or graphs — it takes an $[N, D]$ array of points and returns a description of its shape.

WHY PUT TOPOLOGY AT THE BOTTOM OF THE STACK?

Because persistent homology genuinely does not need to know anything else. It consumes a point cloud and produces a persistence diagram. Placing it at layer zero, with no internal imports, makes that independence structural rather than a matter of discipline — the layering enforces it, and a future change that made the topology code aware of transformers would show up as a new import in a file that has none.

Exploring instead of descending

Gradient descent finds a point. Topology needs a cloud. That mismatch is the reason System 2 exists at all.

System 2 asks a question no ordinary training loop asks: where on this energy surface is the model uncertain? It then deliberately walks there and records the path.

Blank space	$H(z) = \nabla^2_z E(z), \quad \text{blank}(z) \iff \lambda_{\min}(H)$
Langevin	$z_{\{t+1\}} = z_t - \eta \nabla_z E(z_t) + \sqrt{(2\eta/\beta)} \cdot \epsilon$
Temperature	$\beta(z) = \beta_0 / (1 + \text{blank_score}(z))$
Fisher metric	$F(z) = (1/\sigma^2) J^T J, \quad J = \partial p / \partial z \text{ by finite}$

WHY A LANGEVIN WALK RATHER THAN PLAIN GRADIENT DESCENT?

Because descent converges to a point, and a point has no topology. Run gradient descent from a hundred starting positions on a smooth landscape and you get a hundred nearly-identical endpoints. There is nothing there for persistent homology to measure — no loops, no voids, no structure at all.

Adding calibrated noise turns the descent into a diffusion that samples from the energy distribution rather than minimising it. The walker lingers in low-energy basins, occasionally escapes them, and traces out the shape of the region it can reach. That traced shape is the signal. Without the noise term, System 3 would have nothing to work with.

WHY TURN THE NOISE UP WHERE THE MODEL IS UNCERTAIN?

Because those are the regions where new structure might be. A region the model already predicts confidently has nothing to teach it — walking there produces trajectories that trace out shapes the predictor already handles. The interesting geometry is where the model has no opinion.

So the same score that flags a region as blank is fed directly into the temperature map: $\beta = \beta_0 / (1 + \text{blank_score})$. High blank score, low β , large noise term, wider steps. Detection and exploration are not two stages — they are one mechanism with the output of the first wired into the second.

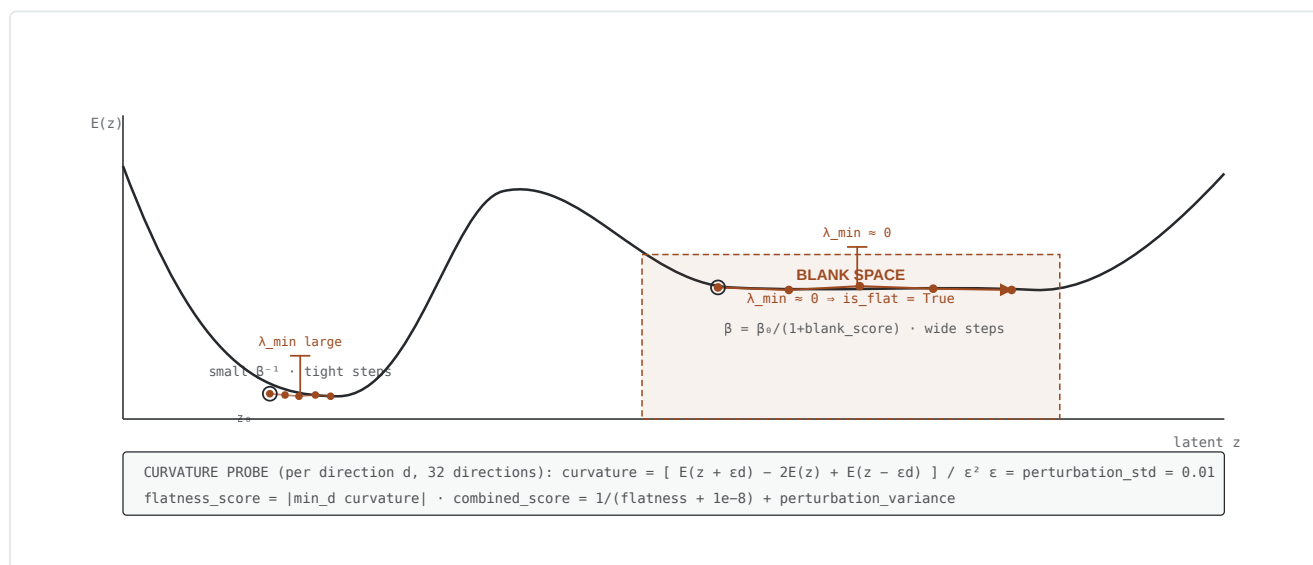


FIGURE 6 The mechanism in one cross-section. Small steps inside the confident well on the left; wide steps across the flat plateau on the right, because the plateau is where the model has no curvature and therefore no opinion.

Detecting blankness

Two independent signals, OR-ed together. The Hessian route probes curvature: for each of 32 random unit directions, a three-point stencil estimates the directional second derivative. If the smallest curvature found is near zero, the surface is locally flat. The perturbation route jitters the input ten times and measures the variance of the predictor's output.

WHY TWO SIGNALS INSTEAD OF ONE?

They catch different failures. Flat curvature means the model has no *preference* — many nearby latents produce the same energy, so it is not committing to anything. High output variance means the model is *unstable* — small input changes produce large output changes, so it is committing, but inconsistently.

A region can be either without being the other, and both are reasons to go look. They are combined with OR for detection and summed for the temperature score.

WHY FINITE DIFFERENCES RATHER THAN AUTOGRAD FOR THE HESSIAN?

Two reasons. Cost: exact second derivatives through a transformer are expensive and the probe runs on up to 256 points per cycle. And correctness of assumption: the energy function is supplied by the caller and may not be twice differentiable, or differentiable at all. A finite difference stencil only requires that you can *evaluate* it.

That is why the whole detector runs under `@torch.no_grad()` — it never needs a graph. It costs 2×32 energy evaluations per point, which is also why exploration is capped at 256 points regardless of batch size.

tcd_jepa/exploration/blank_space_detector.py

lines 38–144

```
38     @torch.no_grad()
39     def compute_energy_hessian_spectrum(
40         self,
41         z: torch.Tensor,
42         energy_fn: callable,
43     ) -> dict[str, torch.Tensor]:
44         """Approximate Hessian eigenvalues via finite differences along random directions.
45
46         H_dd approx (E(z+eps*d) - 2*E(z) + E(z-eps*d)) / eps^2
47
48         Args:
49             z: Latent points [B, D].
50             energy_fn: Maps [B, D] -> [B] energy per sample.
51
52         Returns:
53             Dict with eigenvalue estimates and curvatures.
54         """
55         B, D = z.shape
56         eps = self.perturbation_std
57         e_z = energy_fn(z)
```

```

58
59     num_dirs = min(D, self.num_hessian_directions)
60     directions = torch.randn(num_dirs, D, device=z.device)
61     directions = directions / (directions.norm(dim=-1, keepdim=True) + 1e-8)
62
63     curvatures = []
64     for i in range(num_dirs):
65         d_i = directions[i].unsqueeze(0)
66         e_plus = energy_fn(z + eps * d_i)
67         e_minus = energy_fn(z - eps * d_i)
68         curvature = (e_plus - 2.0 * e_z + e_minus) / (eps ** 2)
69         curvatures.append(curvature)
70
71     curvatures = torch.stack(curvatures, dim=-1) # [B, num_dirs]
72
73     return {
74         "eigenvalues_min": curvatures.min(dim=-1).values,
75         "eigenvalues_max": curvatures.max(dim=-1).values,
76         "trace": curvatures.sum(dim=-1),
77         "curvatures": curvatures,
78     }
79
80 @torch.no_grad()
81 def compute_perturbation_variance(
82     self,
83     z: torch.Tensor,
84     predictor_fn: callable,
85 ) -> torch.Tensor:
86     """Measure predictor output variance under small input perturbations.
87
88     Args:
89         z: Latent points [B, D].
90         predictor_fn: Maps [B, D] -> [B, D_out].
91
92     Returns:
93         Per-sample variance [B].
94     """
95     outputs = []
96     for _ in range(self.num_perturbations):
97         noise = torch.randn_like(z) * self.perturbation_std
98         outputs.append(predictor_fn(z + noise))
99
100     stacked = torch.stack(outputs, dim=0) # [K, B, D_out]
101     return stacked.var(dim=0).mean(dim=-1) # [B]
102
103 @torch.no_grad()
104 def detect(
105     self,
106     z: torch.Tensor,
107     energy_fn: callable,
108     predictor_fn: Optional[callable] = None,
109 ) -> dict[str, torch.Tensor]:
110     """Detect blank spaces using Hessian flatness and perturbation variance.
111
112     Args:
113         z: Latent points [B, D].
114         energy_fn: Maps [B, D] -> [B] energy.
115         predictor_fn: Optional, maps [B, D] -> [B, D_out].
116
117     Returns:
118         Dict with 'is_blank', 'flatness_score', 'variance_score', 'combined_score'.
119     """
120     B = z.shape[0]
121     device = z.device
122
123     hessian_info = self.compute_energy_hessian_spectrum(z, energy_fn)
124     flatness_score = hessian_info["eigenvalues_min"].abs()
125     is_flat = flatness_score < self.flatness_threshold
126
127     if predictor_fn is not None:
128         variance_score = self.compute_perturbation_variance(z, predictor_fn)
129         is_high_var = variance_score > self.variance_threshold
130     else:
131         variance_score = torch.zeros(B, device=device)

```

```

132         is_high_var = torch.zeros(B, dtype=torch.bool, device=device)
133
134     is_blank = is_flat | is_high_var
135     flatness_normalized = 1.0 / (flatness_score + 1e-8)
136     combined_score = flatness_normalized + variance_score
137
138     return {
139         "is_blank": is_blank,
140         "flatness_score": flatness_score,
141         "variance_score": variance_score,
142         "combined_score": combined_score,
143         "hessian_info": hessian_info,
144     }

```

Line 135 inverts flatness before combining, so a *flat* region produces a *large* score — which then becomes a large exploration temperature.

The walk itself

tcd_jepa/exploration/langevin.py

lines 47–162

```

47     def _compute_energy_gradient(
48         self,
49         z: torch.Tensor,
50         energy_fn: callable,
51     ) -> torch.Tensor:
52         """Compute gradient of energy w.r.t. latent positions.
53
54         Args:
55             z: Latent points [B, D], requires_grad will be set.
56             energy_fn: Maps [B, D] -> scalar energy.
57
58         Returns:
59             Gradient [B, D].
60         """
61         # Enable grad even if called from a no_grad context
62         with torch.enable_grad():
63             z_var = z.detach().requires_grad_(True)
64             energy = energy_fn(z_var)
65             if energy.dim() > 0:
66                 energy = energy.sum()
67
68             # Guard: check energy is finite before computing gradients
69             if not torch.isfinite(energy):
70                 logger.warning("Non-finite energy detected in Langevin gradient – returning zero gradient")
71                 return torch.zeros_like(z)
72
73             try:
74                 grad_tuple = torch.autograd.grad(energy, z_var, allow_unused=True)
75             except RuntimeError:
76                 logger.warning("Energy function is not differentiable – returning zero gradient")
77                 return torch.zeros_like(z)
78
79             if grad_tuple[0] is None:
80                 logger.warning("Energy function returned no gradient – returning zero gradient")
81                 return torch.zeros_like(z)
82             grad = grad_tuple[0]
83
84             # Guard: check for NaN gradients
85             if torch.isnan(grad).any():
86                 logger.warning("NaN gradient in Langevin dynamics – returning zero gradient")
87                 return torch.zeros_like(z)
88
89             # Clip gradients for stability
90             grad_norm = grad.norm(dim=-1, keepdim=True).clamp(min=1e-8)
91             grad = grad * torch.clamp(self.grad_clip / grad_norm, max=1.0)
92             return grad.detach()
93
94     def step(
95         self,

```

```

96         z: torch.Tensor,
97         energy_fn: callable,
98         temperature_map: Optional[torch.Tensor] = None,
99     ) -> torch.Tensor:
100         """Take one Langevin dynamics step.
101
102          $z_{t+1} = z_t - \eta * \text{grad}_z E(z_t) + \sqrt{2\eta/\beta} * \epsilon$ 
103
104         Args:
105             z: Current latent positions [B, D].
106             energy_fn: Maps [B, D] -> energy.
107             temperature_map: Per-sample temperature [B]. Lower = more exploration.
108
109         Returns:
110             Updated positions [B, D].
111         """
112         grad = self._compute_energy_gradient(z, energy_fn)
113
114         if temperature_map is not None:
115             beta = temperature_map.unsqueeze(-1) # [B, 1]
116         else:
117             beta = torch.full((z.shape[0], 1), self.temperature, device=z.device)
118
119         beta = beta.clamp(min=self.min_temperature)
120
121         noise_scale = torch.sqrt(2.0 * self.step_size / beta).clamp(max=10.0)
122         noise = self._randn_like(z) * noise_scale
123
124         z_new = z - self.step_size * grad + noise
125         return z_new.detach()
126
127     @torch.no_grad()
128     def sample_trajectory(
129         self,
130         z_init: torch.Tensor,
131         energy_fn: callable,
132         num_steps: Optional[int] = None,
133         blank_score: Optional[torch.Tensor] = None,
134     ) -> torch.Tensor:
135         """Run Langevin dynamics for multiple steps, recording the trajectory.
136
137         Args:
138             z_init: Starting positions [B, D].
139             energy_fn: Energy function.
140             num_steps: Steps to take (defaults to self.max_steps).
141             blank_score: Per-sample blank space score [B] – higher = lower temperature.
142
143         Returns:
144             Trajectory tensor [T, B, D] where T = num_steps + 1.
145         """
146         num_steps = num_steps or self.max_steps
147
148         # Convert blank scores to temperature map: high blank_score -> low temperature
149         if blank_score is not None:
150             # Inverse relationship: more blank -> more exploration (lower beta)
151             temperature_map = self.temperature / (1.0 + blank_score)
152         else:
153             temperature_map = None
154
155         trajectory = [z_init.detach().clone()]
156         z = z_init.detach().clone()
157
158         for _ in range(num_steps):
159             z = self.step(z, energy_fn, temperature_map)
160             trajectory.append(z.clone())
161
162         return torch.stack(trajectory, dim=0) # [T, B, D]

```

Four separate guards around one gradient computation.

WHY FOUR FAILURE GUARDS AROUND A GRADIENT?

Because the energy function is caller-supplied and the sampler has no way to validate it in advance. Each guard corresponds to a real way an arbitrary callable can fail: it can return non-finite values, it can be non-differentiable and raise, it can be differentiable but disconnected from its input and return `None`, and it can return NaN gradients.

All four paths return a zero gradient rather than raising. The walk then degrades to pure diffusion for that step — still producing a trajectory, still usable by System 3 — instead of killing a training run that was otherwise healthy. The alternative, letting exploration crash the run, trades a diagnostic subsystem for the whole job.

Two smaller decisions in the same file. Line 62 re-enables gradients inside a `@torch.no_grad()` context, because `sample_trajectory` is decorated but the gradient helper genuinely needs a graph. And line 121 clamps the noise scale at 10: a near-zero temperature otherwise produces an unbounded jump that throws the walker somewhere meaningless.

The differentiable variant

A second pathway exists for end-to-end training. `differentiable_sample` runs a number of detached warm-up steps to get somewhere interesting cheaply, then a handful of steps with `create_graph=True` so gradients flow back through the exploration itself.

tcd_jepa/exploration/langevin.py

lines 164–249

```
164 def differentiable_step(
165     self,
166     z: torch.Tensor,
167     energy_fn: callable,
168     temperature_map: Optional[torch.Tensor] = None,
169 ) -> torch.Tensor:
170     """Take one differentiable Langevin step using create_graph=True.
171
172     Unlike `step()`, this preserves the computation graph so gradients
173     can flow back through the Langevin dynamics for end-to-end training.
174     Uses the reparameterization trick for the noise term.
175
176     Args:
177         z: Current latent positions [B, D], must have requires_grad=True.
178         energy_fn: Maps [B, D] -> energy (must be differentiable).
179         temperature_map: Per-sample temperature [B].
180
181     Returns:
182         Updated positions [B, D] with gradient graph intact.
183     """
184     with torch.enable_grad():
```

```

185         z_var = z if z.requires_grad else z.requires_grad_(True)
186         energy = energy_fn(z_var)
187         if energy.dim() > 0:
188             energy = energy.sum()
189         grad = torch.autograd.grad(energy, z_var, create_graph=True)[0]
190
191         # Clip gradients for stability
192         grad_norm = grad.norm(dim=-1, keepdim=True).clamp(min=1e-8)
193         grad = grad * torch.clamp(self.grad_clip / grad_norm, max=1.0)
194
195         if temperature_map is not None:
196             beta = temperature_map.unsqueeze(-1)
197         else:
198             beta = torch.full((z.shape[0], 1), self.temperature, device=z.device)
199         beta = beta.clamp(min=self.min_temperature)
200
201         # Reparameterization trick: noise is detached but scale is differentiable
202         noise_scale = torch.sqrt(2.0 * self.step_size / beta)
203         noise = self._randn_like(z).detach() * noise_scale
204
205         z_new = z - self.step_size * grad + noise
206         return z_new
207
208     def differentiable_sample(
209         self,
210         z_init: torch.Tensor,
211         energy_fn: callable,
212         num_warmup_steps: int = 10,
213         num_diff_steps: Optional[int] = None,
214         blank_score: Optional[torch.Tensor] = None,
215     ) -> torch.Tensor:
216         """Run detached warmup followed by K differentiable final steps.
217
218         This enables end-to-end gradient flow through the final exploration
219         steps while keeping the warmup phase detached for efficiency.
220
221         Args:
222             z_init: Starting positions [B, D].
223             energy_fn: Energy function (must support create_graph=True).
224             num_warmup_steps: Number of initial detached warmup steps.
225             num_diff_steps: Number of final differentiable steps (default 5).
226             blank_score: Per-sample blank space score [B].
227
228         Returns:
229             Final positions [B, D] with gradient graph from diff steps.
230         """
231         num_diff_steps = num_diff_steps or 5
232
233         # Temperature map from blank scores
234         if blank_score is not None:
235             temperature_map = self.temperature / (1.0 + blank_score)
236         else:
237             temperature_map = None
238
239         # Phase 1: Detached warmup (no gradient tracking)
240         z = z_init.detach().clone()
241         for _ in range(num_warmup_steps):
242             z = self.step(z, energy_fn, temperature_map)
243
244         # Phase 2: Differentiable final steps
245         z = z.requires_grad_(True)
246         for _ in range(num_diff_steps):
247             z = self.differentiable_step(z, energy_fn, temperature_map)
248
249         return z

```

The noise is detached; its scale is not. Reparameterisation, applied to a sampler.

WHY WARM UP DETACHED AND ONLY DIFFERENTIATE THE LAST FEW STEPS?

Because keeping the graph is what costs memory, and the early steps are the least informative. The first ten steps mostly transport the walker from wherever it started to the region it will explore; gradients through that transport tell you little. The last five steps are where it is actually sampling the region of interest, and those are the ones worth attaching to the loss.

Fisher information, and why there is a third signal

tcd_jepa/exploration/fisher_metric.py

lines 33-139

```
33 def compute_jacobian_fd(
34     self,
35     z: torch.Tensor,
36     predictor_fn: callable,
37 ) -> torch.Tensor:
38     """Estimate the Jacobian dp/dz via finite differences.
39
40     Args:
41         z: Latent points [B, D_in].
42         predictor_fn: Maps [B, D_in] -> [B, D_out].
43
44     Returns:
45         Jacobian estimate [B, D_out, D_in] (sampled directions).
46     """
47     B, D_in = z.shape
48     eps = self.finite_diff_eps
49
50     # Sample random directions to estimate Jacobian columns
51     num_dirs = min(D_in, self.num_jacobian_samples)
52     directions = torch.randn(num_dirs, D_in, device=z.device)
53     directions = directions / directions.norm(dim=-1, keepdim=True)
54
55     with torch.no_grad():
56         p_z = predictor_fn(z) # [B, D_out]
57         _ = p_z.shape[-1]
58
59     jacobian_cols = []
60     with torch.no_grad():
61         for i in range(num_dirs):
62             d_i = directions[i].unsqueeze(0) * eps # [1, D_in]
63             p_plus = predictor_fn(z + d_i)
64             dp = (p_plus - p_z) / eps
65             jacobian_cols.append(dp)
66
67     # [B, D_out, num_dirs]
68     return torch.stack(jacobian_cols, dim=-1)
69
70 def compute_fisher_matrix(
71     self,
72     z: torch.Tensor,
73     predictor_fn: callable,
74 ) -> torch.Tensor:
75     """Compute the Fisher information matrix at latent points.
76
77     
$$F(z) = (1/\sigma^2) * J^T J$$

78
79     Args:
```

```

80         z: Latent points [B, D_in].
81         predictor_fn: Maps [B, D_in] -> [B, D_out].
82
83     Returns:
84         Fisher matrix [B, K, K] where K = num_jacobian_samples.
85     """
86     J = self.compute_jacobian_fd(z, predictor_fn) # [B, D_out, K]
87     fisher = torch.bmm(J.transpose(1, 2), J) / self.noise_variance
88     return fisher
89
90     def compute_geodesic_distance(
91         self,
92         z1: torch.Tensor,
93         z2: torch.Tensor,
94         predictor_fn: callable,
95     ) -> torch.Tensor:
96         """Approximate geodesic distance between points using the Fisher metric.
97
98         Uses the midpoint Fisher matrix:  $d(z1, z2) \approx \sqrt{(z2-z1)^T F(z_{mid}) (z2-z1)}$ 
99
100     Args:
101         z1, z2: Latent points [B, D].
102         predictor_fn: Predictor function.
103
104     Returns:
105         Geodesic distances [B].
106     """
107     z_mid = (z1 + z2) / 2.0
108     J = self.compute_jacobian_fd(z_mid, predictor_fn) # [B, D_out, K]
109
110     # Project difference onto Jacobian directions
111     # Since J is in sampled directions, project diff onto those directions
112     fisher = torch.bmm(J.transpose(1, 2), J) / self.noise_variance # [B, K, K]
113
114     # For the actual distance, we need diff in the sampled basis
115     # Approximate: use the trace of Fisher as a scalar metric
116     fisher_trace = torch.diagonal(fisher, dim1=-2, dim2=-1).sum(dim=-1) # [B]
117     euclidean_dist = (z2 - z1).pow(2).sum(dim=-1) # [B]
118
119     return torch.sqrt(fisher_trace * euclidean_dist / z1.shape[-1])
120
121     def compute_metric_tensor_trace(
122         self,
123         z: torch.Tensor,
124         predictor_fn: callable,
125     ) -> torch.Tensor:
126         """Compute trace of Fisher metric (scalar curvature proxy).
127
128         Higher trace = predictor is more sensitive at this point = well-mapped region.
129         Lower trace = predictor is insensitive = potential blank space.
130
131     Args:
132         z: Latent points [B, D].
133         predictor_fn: Predictor function.
134
135     Returns:
136         Fisher trace per sample [B].
137     """
138     fisher = self.compute_fisher_matrix(z, predictor_fn)
139     return torch.diagonal(fisher, dim1=-2, dim2=-1).sum(dim=-1)

```

Note the honest comment at lines 110–114: the geodesic distance is a trace-scaled Euclidean approximation, not a true geodesic integral.

WHY A RIEMANNIAN METRIC ON TOP OF TWO EXISTING UNCERTAINTY SIGNALS?

Because Euclidean distance in latent space is not the distance the model experiences. Two latents a fixed distance apart may produce nearly identical predictions in one region and wildly different ones in another. The Fisher metric measures that: it is the sensitivity of the predictor's output to movement, which is the geometry that actually matters for exploration.

In practice the code uses only its trace, reported as

`mean_fisher_trace` — a scalar per point, high where the predictor is sensitive and low where it is not. It is a diagnostic rather than a driver: the temperature map is built from the blank-space score alone. The full metric machinery is present and tested but the loop does not yet steer with it.

Recording the paths

tcd_jepa/exploration/trajectory_tracker.py

lines 30-91

```
30 def add_trajectory(
31     self,
32     z_trajectory: torch.Tensor,
33     energies: Optional[torch.Tensor] = None,
34     metadata: Optional[dict] = None,
35 ) -> None:
36     """Store a new exploration trajectory.
37
38     Args:
39         z_trajectory: Trajectory points [T, B, D] or [T, D].
40         energies: Optional energy values at each point [T, B] or [T].
41         metadata: Optional dict with extra info (epoch, blank_scores, etc.).
42     """
43     # Flatten batch dimension if present: [T, B, D] -> [T*B, D]
44     if z_trajectory.dim() == 3:
45         T, B, D = z_trajectory.shape
46         z_flat = z_trajectory.reshape(T * B, D)
47     else:
48         z_flat = z_trajectory
49
50     # Subsample if too many points
51     if z_flat.shape[0] > self.max_points_per_trajectory:
52         indices = torch.linspace(
53             0, z_flat.shape[0] - 1, self.max_points_per_trajectory
54         ).long()
55         z_flat = z_flat[indices]
56
57     self._trajectories.append(z_flat.detach().cpu())
58
59     if energies is not None:
60         self._energies.append(energies.detach().cpu().flatten())
61     else:
62         self._energies.append(None)
63
```

```

64     self._metadata.append(metadata or {})
65
66     # Evict oldest if over capacity
67     if len(self._trajectories) > self.max_trajectories:
68         self._trajectories.pop(0)
69         self._energies.pop(0)
70         self._metadata.pop(0)
71
72     def get_point_cloud(
73         self,
74         last_n: Optional[int] = None,
75     ) -> torch.Tensor:
76         """Get combined point cloud from recent trajectories.
77
78         Args:
79             last_n: Use only the last N trajectories. None = all.
80
81         Returns:
82             Point cloud [N_total, D].
83         """
84         if not self._trajectories:
85             raise ValueError("No trajectories recorded yet")
86
87         trajectories = self._trajectories
88         if last_n is not None:
89             trajectories = trajectories[-last_n:]
90
91         return torch.cat(trajectories, dim=0)

```

A ring buffer with two caps and a deliberate loss of information.

WHY THROW AWAY THE TIME ORDERING?

Because persistent homology does not use it, and pretending otherwise would be misleading. A trajectory arrives as $[T, B, D]$ and is immediately flattened to $[T \cdot B, D]$ — a bag of points with no notion of which came first.

This is the right call for what System 3 does. Homology measures the shape of the *set* of points: whether they form clusters, whether they enclose a loop, whether they surround a void. A loop in the point set means the walker's reachable region has a hole in it, which is a fact about the energy landscape. Whether the walker went round it clockwise is not.

Two caps: at most 1000 points per trajectory, subsampled by `linspace` so the whole path is represented rather than just its beginning; and at most 100 trajectories, oldest evicted first. Everything is moved to CPU on arrival, which keeps a long run from silently accumulating GPU memory in a buffer nobody is watching.

tcd_jepa/core/system2_explorer.py

lines 58–133

```

58     def explore(
59         self,
60         z_context: torch.Tensor,

```

```

61         energy_fn: callable,
62         predictor_fn: Optional[callable] = None,
63         num_langevin_steps: Optional[int] = None,
64         epoch: Optional[int] = None,
65     ) -> dict[str, torch.Tensor]:
66         """Run one exploration cycle.
67
68     Args:
69         z_context: Encoded latent representations [B, N, D] or [B, D].
70         energy_fn: Maps [B, D] -> [B] energy.
71         predictor_fn: Optional predictor for variance detection.
72         num_langevin_steps: Override default step count.
73         epoch: Current epoch for metadata.
74
75     Returns:
76         Dict with exploration results:
77         'trajectory': [T, B_flat, D]
78         'blank_info': blank space detection results
79         'fisher_trace': Fisher metric trace at starting points
80     """
81     # Flatten spatial dims if needed: [B, N, D] -> [B*N, D]
82     if z_context.dim() == 3:
83         B, N, D = z_context.shape
84         z_flat = z_context.reshape(B * N, D)
85     else:
86         z_flat = z_context
87         D = z_flat.shape[-1]
88
89     # Subsample for efficiency (explore a subset of points)
90     max_explore = min(z_flat.shape[0], 256)
91     if z_flat.shape[0] > max_explore:
92         indices = torch.randperm(z_flat.shape[0], device=z_flat.device)[:max_explore]
93         z_explore = z_flat[indices]
94     else:
95         z_explore = z_flat
96
97     # Step 1: Detect blank spaces
98     blank_info = self.blank_detector.detect(
99         z_explore, energy_fn, predictor_fn
100 )
101
102     # Step 2: Run Langevin dynamics biased toward blank regions
103     trajectory = self.langevin.sample_trajectory(
104         z_explore,
105         energy_fn,
106         num_steps=num_langevin_steps,
107         blank_score=blank_info["combined_score"],
108     )
109
110     # Step 3: Compute Fisher metric at starting points
111     if predictor_fn is not None:
112         fisher_trace = self.fisher.compute_metric_tensor_trace(
113             z_explore, predictor_fn
114         )
115     else:
116         fisher_trace = torch.zeros(z_explore.shape[0], device=z_explore.device)
117
118     # Step 4: Record trajectory
119     self.trajectory_tracker.add_trajectory(
120         trajectory,
121         metadata={"epoch": epoch},
122     )
123
124     return {
125         "trajectory": trajectory,
126         "blank_info": blank_info,
127         "fisher_trace": fisher_trace,
128         "z_explored": z_explore,
129     }
130
131 def get_point_cloud(self, last_n: Optional[int] = None) -> torch.Tensor:
132     """Get accumulated point cloud for System 3."""
133     return self.trajectory_tracker.get_point_cloud(last_n=last_n)

```

The orchestrator: detect, walk, measure, record.

Measuring shape

System 3 takes the cloud of points a random walk left behind, measures its shape, and turns each durable shape into a neural network. This is where the claim from Part I becomes a mechanism.

The instrument is persistent homology. Grow a ball of radius ϵ around every point and let the balls merge. Watch which topological features appear and disappear as ϵ grows from zero upward. A feature that survives across a wide range of ϵ is real structure; one that appears and dies immediately is sampling noise. That survival span is called **persistence**, and it is the only quantity that decides whether a module gets built.

WHY PERSISTENT HOMOLOGY RATHER THAN CLUSTERING?

Two reasons, and the second is the important one.

First, clustering only finds clusters. In homology terms that is H_0 — connected components — and it is one third of what is available. Loops (H_1) and enclosed voids (H_2) are invisible to every clustering algorithm, and they are exactly the structures that indicate something more interesting than "these points are near each other."

Second, and more fundamental: clustering requires you to choose a scale. Pick a distance threshold, or a k , and the answer changes with the choice. Persistent homology refuses the choice — it computes the answer at *every* scale simultaneously and then reports which features were stable across scales. The noise filter is not a hyperparameter; it falls out of the method.

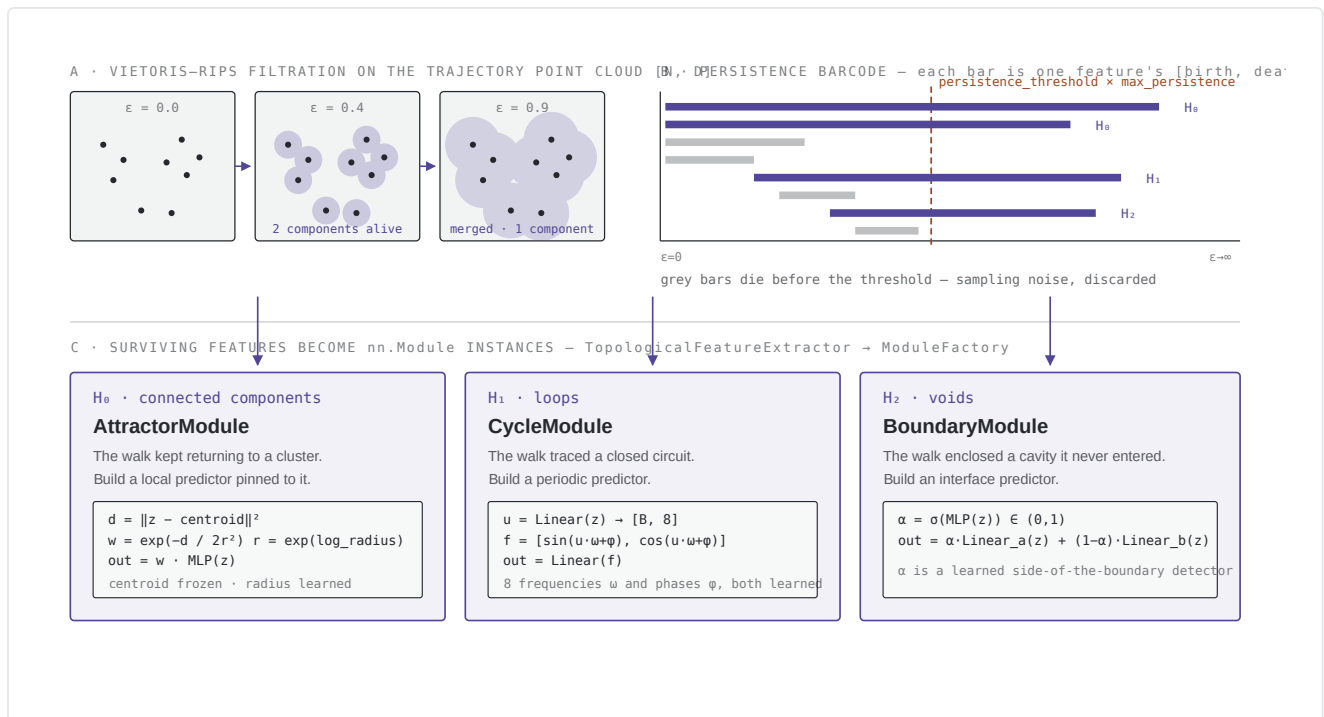


FIGURE 7 Crystallization end to end. The dimension of the surviving homology class picks the module *class*; the birth and death scales become its parameters. Nothing about this mapping is learned — it is a fixed vocabulary of three shapes, and the data decides how many of each get built.

Computing it

Three backends, tried in order: `giotto-tda`, then `ripser`, then a `scipy` single-linkage fallback.

```
tcd_jepa/topology/persistent_homology.py lines 20-160

20 def _get_backend():
21     """Detect available persistent homology backend."""
22     try:
23         from gtdd.homology import VietorisRipsPersistence # noqa: F401
24         return "giotto"
25     except ImportError:
26         pass
27     try:
28         from ripser import ripser # noqa: F401
29         return "ripser"
30     except ImportError:
31         pass
32     return None
33
34
35 class PersistentHomologyComputer:
36     """Computes persistent homology on point clouds from exploration trajectories.
37
38     Wraps giotto-tda or ripser to compute Vietoris-Rips persistence diagrams.
39     """
40
41     def __init__(
42         self,
43         max_homology_dim: int = 2,
44         max_edge_length: float = float("inf"),
45         n_jobs: int = 1,
46     ) -> None:
47         self.max_homology_dim = max_homology_dim
48         self.max_edge_length = max_edge_length
49         self.n_jobs = n_jobs
```

```

50         self._backend = _get_backend()
51
52     @property
53     def backend(self) -> Optional[str]:
54         return self._backend
55
56     def compute(
57         self,
58         point_cloud: torch.Tensor,
59         max_points: int = 500,
60     ) -> dict[str, np.ndarray]:
61         """Compute persistent homology of a point cloud.
62
63         Args:
64             point_cloud: Points [N, D].
65             max_points: Subsample if point cloud is larger (PH is  $O(N^3)$ ).
66
67         Returns:
68             Dict with:
69                 'diagram': persistence diagram array [K, 3] (birth, death, dim)
70                 'H0', 'H1', 'H2': filtered diagrams per dimension
71         """
72         points = point_cloud.detach().cpu().numpy()
73
74         # Subsample for computational feasibility (seeded for reproducibility)
75         if points.shape[0] > max_points:
76             rng = np.random.RandomState(42)
77             indices = rng.choice(points.shape[0], max_points, replace=False)
78             points = points[indices]
79
80         if self._backend == "giotto":
81             return self._compute_giotto(points)
82         elif self._backend == "ripser":
83             return self._compute_ripser(points)
84         else:
85             logger.warning(
86                 "No PH backend (giotto-tda/ripser) installed – using scipy fallback "
87                 "(only H0 via single-linkage). Install giotto-tda or ripser for full PH."
88             )
89             return self._compute_fallback(points)
90
91     def _compute_giotto(self, points: np.ndarray) -> dict[str, np.ndarray]:
92         """Compute using giotto-tda."""
93         from gtda.homology import VietorisRipsPersistence
94
95         vr = VietorisRipsPersistence(
96             homology_dimensions=list(range(self.max_homology_dim + 1)),
97             max_edge_length=self.max_edge_length,
98             n_jobs=self.n_jobs,
99         )
100         # giotto expects [n_samples, n_points, n_dims]
101         diagrams = vr.fit_transform(points[np.newaxis])[0]
102
103         result = {"diagram": diagrams}
104         for dim in range(self.max_homology_dim + 1):
105             mask = diagrams[:, 2] == dim
106             result[f"H{dim}"] = diagrams[mask][:, :2]
107         return result
108
109     def _compute_ripser(self, points: np.ndarray) -> dict[str, np.ndarray]:
110         """Compute using ripser."""
111         from ripser import ripser
112
113         result_ripser = ripser(
114             points,
115             maxdim=self.max_homology_dim,
116             thresh=self.max_edge_length if self.max_edge_length != float("inf") else np.inf,
117         )
118
119         # Convert to unified format
120         all_features = []
121         result = {}
122         for dim, dgm in enumerate(result_ripser["dgms"]):
123             # Filter infinite deaths

```

```

124         finite_mask = np.isfinite(dgm[:, 1])
125         dgm_finite = dgm[finite_mask]
126         result[f"H{dim}"] = dgm_finite
127
128         dims_col = np.full((dgm_finite.shape[0], 1), dim)
129         all_features.append(np.hstack([dgm_finite, dims_col]))
130
131     if all_features:
132         result["diagram"] = np.vstack(all_features)
133     else:
134         result["diagram"] = np.empty((0, 3))
135     return result
136
137 def _compute_fallback(self, points: np.ndarray) -> dict[str, np.ndarray]:
138     """Approximate PH using pairwise distances when no backend is available.
139
140     Computes  $H_0$  via single-linkage clustering (connected components).
141     """
142     from scipy.cluster.hierarchy import linkage
143     from scipy.spatial.distance import pdist
144
145     dists = pdist(points)
146     Z = linkage(dists, method="single")
147
148     #  $H_0$ : connected components merge at distances given by linkage
149     births = np.zeros(len(Z))
150     deaths = Z[:, 2]
151     h0 = np.column_stack([births, deaths])
152
153     diagram = np.column_stack([births, deaths, np.zeros(len(births))])
154
155     return {
156         "diagram": diagram,
157         "H0": h0,
158         "H1": np.empty((0, 2)),
159         "H2": np.empty((0, 2)),
160     }

```

Backend detection at import time, then three implementations normalised to one output format. Ripser's infinite-death features are dropped at line 124 — a component that never dies carries no finite persistence.

WHY KEEP AN H_0 -ONLY FALLBACK THAT CANNOT DO THE INTERESTING PART?

Because failing soft is better than failing hard for an optional dependency — but this one has a sharp edge worth knowing about.

Single-linkage clustering genuinely *is* the H_0 filtration; the fallback is mathematically correct as far as it goes. What it cannot do is loops or voids. So a run without `giotto-tda` or `ripser` installed will keep going, log one warning, and never crystallize anything but attractor modules. If you see only `mod_attractor_*` ids and wonder why no cycles are forming, check the backend before you touch any threshold.

WHY SUBSAMPLE TO 500 POINTS?

Vietoris–Rips is roughly $O(N^3)$. A point cloud from five trajectories can easily be tens of thousands of points, which is the difference between a crystallization step that takes a second and one that takes an hour. The subsample is seeded (`RandomState(42)`) so the result is reproducible rather than varying run to run.

Deciding what counts

tcd_jepa/topology/persistence_diagrams.py

lines 30–135

```
30 def compute_persistence(self, diagram: np.ndarray) -> np.ndarray:
31     """Compute persistence (death - birth) for each feature.
32
33     Args:
34         diagram: Persistence diagram [K, 2] with (birth, death) pairs.
35
36     Returns:
37         Persistence values [K].
38     """
39     if len(diagram) == 0:
40         return np.array([])
41     return diagram[:, 1] - diagram[:, 0]
42
43 def filter_by_persistence(
44     self,
45     diagram: np.ndarray,
46     threshold: Optional[float] = None,
47 ) -> tuple[np.ndarray, np.ndarray]:
48     """Filter features by persistence threshold.
49
50     Args:
51         diagram: [K, 2] persistence diagram.
52         threshold: Override default threshold.
53
54     Returns:
55         (filtered_diagram, indices) – surviving features and their original indices.
56     """
57     if len(diagram) == 0:
58         return diagram, np.array([], dtype=int)
59
60     persistence = self.compute_persistence(diagram)
61     thresh = threshold or self.persistence_threshold
62
63     if self.relative_threshold:
64         # Threshold relative to max persistence
65         max_pers = persistence.max() if len(persistence) > 0 else 1.0
66         thresh = thresh * max_pers
67
68     mask = persistence >= thresh
69     indices = np.where(mask)[0]
70
71     # Enforce min/max feature counts
72     if len(indices) < self.min_features and len(persistence) > 0:
73         # Take top-k by persistence
74         top_k = min(self.min_features, len(persistence))
75         indices = np.argsort(persistence)[-top_k:]
76
77     if len(indices) > self.max_features:
```

```

78         # Keep only top-k
79         pers_at_indices = persistence[indices]
80         top_k_local = np.argsort(pers_at_indices)[-self.max_features:]
81         indices = indices[top_k_local]
82
83         return diagram[indices], indices
84
85     def analyze(
86         self,
87         ph_result: dict[str, np.ndarray],
88     ) -> dict[str, dict]:
89         """Full analysis of a persistence homology result.
90
91         Args:
92             ph_result: Output from PersistentHomologyComputer.compute().
93
94         Returns:
95             Dict per dimension with filtered features and statistics:
96             {
97                 'H0': {'features': [...], 'persistence': [...], 'count': int},
98                 'H1': {...},
99                 'H2': {...},
100             }
101         """
102         result = {}
103         for dim_key in ["H0", "H1", "H2"]:
104             dgm = ph_result.get(dim_key, np.empty((0, 2)))
105             if len(dgm) == 0:
106                 result[dim_key] = {
107                     "features": np.empty((0, 2)),
108                     "persistence": np.array([]),
109                     "count": 0,
110                     "mean_persistence": 0.0,
111                     "max_persistence": 0.0,
112                 }
113                 continue
114
115             filtered, indices = self.filter_by_persistence(dgm)
116             persistence = self.compute_persistence(filtered)
117
118             result[dim_key] = {
119                 "features": filtered,
120                 "persistence": persistence,
121                 "count": len(filtered),
122                 "mean_persistence": float(persistence.mean()) if len(persistence) > 0 else 0.0,
123                 "max_persistence": float(persistence.max()) if len(persistence) > 0 else 0.0,
124             }
125
126         return result
127
128     def total_persistence(self, ph_result: dict[str, np.ndarray]) -> float:
129         """Compute total persistence across all dimensions (landscape summary statistic)."""
130         total = 0.0
131         for dim_key in ["H0", "H1", "H2"]:
132             dgm = ph_result.get(dim_key, np.empty((0, 2)))
133             if len(dgm) > 0:
134                 total += float(self.compute_persistence(dgm).sum())
135         return total

```

The gate. Persistence is death minus birth; the threshold is applied relative to the most persistent feature within each homology dimension.

WHY A RELATIVE THRESHOLD RATHER THAN AN ABSOLUTE ONE?

Because latent norms drift during training. Early on, representations are small and all persistence values are small with them; later they grow. An absolute threshold tuned at epoch 5 finds nothing at epoch 50, or everything — and it would need retuning for every dataset and every embedding dimension.

$0.3 \times \text{max_persistence}$ asks a scale-free question instead: *is this feature comparable to the most durable one we found?* That question means the same thing whether latents have norm 1 or norm 100.

WHY CAP AT TWO FEATURES PER DIMENSION?

Rate limiting. A single crystallization event can otherwise register a dozen modules, all untrained, all immediately influencing the predictor. `max_features=2` bounds how fast the module population can grow, which gives the router and the mixing coefficient time to adapt to each new arrival before the next batch shows up.

Turning features into networks

tcd_jepa/topology/feature_extraction.py

lines 15-125

```
15 @dataclass
16 class TopologicalFeature:
17     """A persistent topological feature extracted from exploration trajectories."""
18     dim: int # Homology dimension (0, 1, or 2)
19     birth: float # Birth time (scale)
20     death: float # Death time (scale)
21     persistence: float # death - birth
22     centroid: Optional[np.ndarray] = None # Representative point in latent space
23     points: Optional[np.ndarray] = None # Points belonging to this feature
24     module_type: str = "" # Assigned module type
25
26
27 class TopologicalFeatureExtractor:
28     """Extracts module-forming features from persistence analysis results.
29
30     Assigns each significant topological feature a module type based
31     on its homology dimension and characteristics.
32     """
33
34     def __init__(
35         self,
36         persistence_threshold: float = 0.3,
37         min_cluster_size: int = 5,
```

```

38     ) -> None:
39         self.persistence_threshold = persistence_threshold
40         self.min_cluster_size = min_cluster_size
41
42     def extract_features(
43         self,
44         ph_analysis: dict[str, dict],
45         point_cloud: Optional[torch.Tensor] = None,
46     ) -> list[TopologicalFeature]:
47         """Extract TopologicalFeature objects from persistence analysis.
48
49         Args:
50             ph_analysis: Output from PersistenceDiagramAnalyzer.analyze().
51             point_cloud: Optional [N, D] point cloud for centroid computation.
52
53         Returns:
54             List of TopologicalFeature candidates for module formation.
55         """
56         features = []
57
58         for dim, dim_key in enumerate(["H0", "H1", "H2"]):
59             info = ph_analysis.get(dim_key, {})
60             dgm = info.get("features", np.empty((0, 2)))
61
62             if len(dgm) == 0:
63                 continue
64
65             for i in range(len(dgm)):
66                 birth, death = dgm[i]
67                 persistence = death - birth
68
69                 # Skip low-persistence features
70                 if persistence < self.persistence_threshold:
71                     continue
72
73                 module_type = {0: "attractor", 1: "cycle", 2: "boundary"}[dim]
74
75                 feat = TopologicalFeature(
76                     dim=dim,
77                     birth=float(birth),
78                     death=float(death),
79                     persistence=float(persistence),
80                     module_type=module_type,
81                 )
82
83                 # Compute centroid if point cloud available
84                 if point_cloud is not None and dim == 0:
85                     feat.centroid = self._compute_cluster_centroid(
86                         point_cloud, birth, death
87                     )
88
89                 features.append(feat)
90
91             # Sort by persistence (most significant first)
92             features.sort(key=lambda f: f.persistence, reverse=True)
93             return features
94
95     def _compute_cluster_centroid(
96         self,
97         point_cloud: torch.Tensor,
98         birth_scale: float,
99         death_scale: float,
100     ) -> np.ndarray:
101         """Estimate the centroid of a cluster corresponding to an H_0 feature.
102
103         Uses the midpoint scale to identify which points belong to the cluster.
104         """
105         points = point_cloud.numpy() if isinstance(point_cloud, torch.Tensor) else point_cloud
106         # Use a random subset as centroid proxy (exact cluster assignment
107         # would require the full filtration which is expensive)
108         mid_scale = (birth_scale + death_scale) / 2.0
109
110         # Rough approach: find dense region at the characteristic scale
111         from scipy.spatial.distance import cdist

```

```

112     N = len(points)
113     if N > 200:
114         idx = np.random.choice(N, 200, replace=False)
115         subset = points[idx]
116     else:
117         subset = points
118
119     dists = cdist(subset, subset)
120     # Count neighbors within mid_scale
121     neighbor_counts = (dists < mid_scale).sum(axis=1)
122     densest = np.argmax(neighbor_counts)
123     nearby = subset[dists[densest] < mid_scale]
124
125     return nearby.mean(axis=0)

```

Line 73 is the entire dimension-to-type vocabulary. Centroids are computed only for H_0 (line 84), because only the attractor module has a location.

tcd_jepa/modules/module_factory.py

lines 17-151

```

17 class AttractorModule(nn.Module):
18     """Local predictor centered on a cluster centroid in latent space.
19
20     Specializes in predicting representations near a specific attractor
21     basin discovered by System 2's exploration.
22     """
23
24     def __init__(self, embed_dim: int, centroid: Optional[torch.Tensor] = None) -> None:
25         super().__init__()
26         self.embed_dim = embed_dim
27         self.centroid = nn.Parameter(
28             centroid if centroid is not None else torch.randn(embed_dim) * 0.01,
29             requires_grad=False,
30         )
31         self.predictor = nn.Sequential(
32             nn.Linear(embed_dim, embed_dim * 2),
33             nn.GELU(),
34             nn.Linear(embed_dim * 2, embed_dim),
35         )
36         # Learnable radius of influence
37         self.log_radius = nn.Parameter(torch.tensor(0.0))
38
39     def forward(self, z: torch.Tensor) -> torch.Tensor:
40         """Predict with attention weight based on distance to centroid.
41
42         Args:
43             z: Input representations [B, D].
44
45         Returns:
46             Weighted prediction [B, D].
47         """
48         dist = (z - self.centroid).pow(2).sum(dim=-1, keepdim=True)
49         radius = self.log_radius.exp()
50         weight = torch.exp(-dist / (2.0 * radius ** 2 + 1e-8))
51         return weight * self.predictor(z)
52
53
54 class CycleModule(nn.Module):
55     """Periodic predictor for cyclic patterns in latent space.
56
57     Captures oscillatory structures discovered as  $H_1$  features.
58     """
59
60     def __init__(self, embed_dim: int, num_frequencies: int = 8) -> None:
61         super().__init__()
62         self.embed_dim = embed_dim
63         self.frequencies = nn.Parameter(torch.randn(num_frequencies) * 0.1)
64         self.phases = nn.Parameter(torch.randn(num_frequencies) * 0.1)
65         self.projection = nn.Linear(embed_dim, num_frequencies)
66         self.output = nn.Linear(num_frequencies * 2, embed_dim)
67

```

```

68     def forward(self, z: torch.Tensor) -> torch.Tensor:
69         """Apply periodic transformation.
70
71         Args:
72             z: Input [B, D].
73
74         Returns:
75             Periodic prediction [B, D].
76         """
77         projected = self.projection(z) # [B, num_freq]
78         sin_features = torch.sin(projected * self.frequencies + self.phases)
79         cos_features = torch.cos(projected * self.frequencies + self.phases)
80         periodic = torch.cat([sin_features, cos_features], dim=-1)
81         return self.output(periodic)
82
83
84     class BoundaryModule(nn.Module):
85         """Interface predictor for boundary structures in latent space.
86
87         Captures surface/boundary structures discovered as H_2 features.
88         """
89
90         def __init__(self, embed_dim: int) -> None:
91             super().__init__()
92             self.embed_dim = embed_dim
93             self.boundary_detector = nn.Sequential(
94                 nn.Linear(embed_dim, embed_dim),
95                 nn.GELU(),
96                 nn.Linear(embed_dim, 1),
97                 nn.Sigmoid(),
98             )
99             self.predictor_a = nn.Linear(embed_dim, embed_dim)
100             self.predictor_b = nn.Linear(embed_dim, embed_dim)
101
102         def forward(self, z: torch.Tensor) -> torch.Tensor:
103             """Predict using boundary-aware interpolation.
104
105             Args:
106                 z: Input [B, D].
107
108             Returns:
109                 Boundary-interpolated prediction [B, D].
110             """
111             alpha = self.boundary_detector(z) # [B, 1]
112             return alpha * self.predictor_a(z) + (1 - alpha) * self.predictor_b(z)
113
114
115     class ModuleFactory:
116         """Creates predictor modules from topological features."""
117
118         def __init__(self, embed_dim: int) -> None:
119             self.embed_dim = embed_dim
120
121         def create_module(
122             self,
123             feature: TopologicalFeature,
124             device: torch.device = torch.device("cpu"),
125         ) -> nn.Module:
126             """Instantiate a predictor module from a topological feature.
127
128             Args:
129                 feature: TopologicalFeature with dim, centroid, etc.
130                 device: Target device.
131
132             Returns:
133                 Initialized nn.Module.
134             """
135             if feature.module_type == "attractor" or feature.dim == 0:
136                 centroid = None
137                 if feature.centroid is not None:
138                     centroid = torch.from_numpy(feature.centroid).float()
139                 module = AttractorModule(self.embed_dim, centroid=centroid)
140
141             elif feature.module_type == "cycle" or feature.dim == 1:

```

```

142         module = CycleModule(self.embed_dim)
143
144     elif feature.module_type == "boundary" or feature.dim == 2:
145         module = BoundaryModule(self.embed_dim)
146
147     else:
148         # Default to attractor
149         module = AttractorModule(self.embed_dim)
150
151     return module.to(device)

```

Three module classes, all deliberately small.

WHY THESE THREE ARCHITECTURES SPECIFICALLY?

Each one is the natural predictor for the shape that produced it.

An **H₀ component** means the walk kept returning to a cluster — a basin in the energy landscape with a location. So

`AttractorModule` is a local predictor with a location: a small MLP multiplied by a Gaussian in the distance to the cluster centroid. Far from the centroid the gate closes and the module contributes nothing. The centroid is frozen (it is a measured fact about the landscape, not something to optimise) while the radius of influence is learned (how far the basin's authority extends is not something the persistence diagram tells you).

An **H₁ loop** means the reachable region has a hole in it — the walk went around something. Periodicity is the natural functional form, so `CycleModule` projects the input onto eight learned frequencies and takes sines and cosines. It is a Fourier feature layer, chosen because a cycle is what a Fourier basis is for.

An **H₂ void** means an enclosed cavity — a region with an inside and an outside and a surface between them. So `BoundaryModule` learns a sigmoid that decides which side of something the input is on, and blends two linear maps accordingly. It is an interface predictor because a void is an interface.

WHY STOP AT THREE? WHY NOT H_3 AND BEYOND?

Cost and meaning, in that order. Homology dimension k requires enumerating $(k+1)$ -simplices, and the count grows combinatorially — H_3 on 500 points is dramatically more expensive than H_2 . And the interpretive story gets thin: it is clear what a predictor shaped like a cluster or a cycle or a boundary should do, and much less clear what one shaped like a three-dimensional cavity in a 192-dimensional space should do.

`max_homology_dim=2` is where the cost curve and the interpretability curve cross.

Lifecycle

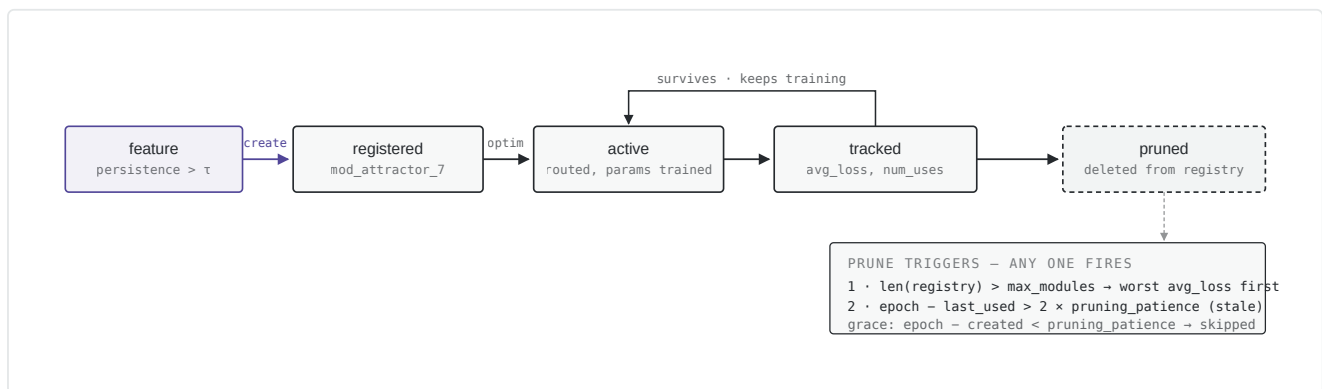


FIGURE 8 Modules are not permanent. The registry tracks average loss per module and prunes on two independent criteria, with a grace period protecting new arrivals.

tcd_jepa/modules/module_registry.py

lines 14-167

```
14 @dataclass
15 class ModuleRecord:
16     """Tracks a registered module's metadata and performance."""
17     module_id: str
18     module: nn.Module
19     feature: TopologicalFeature
20     created_epoch: int
21     total_loss: float = 0.0
22     num_uses: int = 0
23     last_used_epoch: int = 0
24
25     @property
26     def avg_loss(self) -> float:
27         return self.total_loss / max(self.num_uses, 1)
28
29
30 class ModuleRegistry:
31     """Manages dynamically created predictor modules.
32
33     Tracks performance of each module and prunes underperformers.
34     """
```



```

35
36 def __init__(
37     self,
38     max_modules: int = 20,
39     pruning_patience: int = 10,
40     min_improvement: float = 0.01,
41 ) -> None:
42     self.max_modules = max_modules
43     self.pruning_patience = pruning_patience
44     self.min_improvement = min_improvement
45     self._modules: dict[str, ModuleRecord] = {}
46     self._next_id = 0
47
48 def register(
49     self,
50     module: nn.Module,
51     feature: TopologicalFeature,
52     epoch: int,
53 ) -> str:
54     """Register a new module.
55
56     Args:
57         module: The nn.Module to register.
58         feature: Topological feature that spawned it.
59         epoch: Current epoch.
60
61     Returns:
62         Module ID string.
63     """
64     module_id = f"mod_{feature.module_type}_{self._next_id}"
65     self._next_id += 1
66
67     self._modules[module_id] = ModuleRecord(
68         module_id=module_id,
69         module=module,
70         feature=feature,
71         created_epoch=epoch,
72         last_used_epoch=epoch,
73     )
74
75     # Prune if over capacity
76     if len(self._modules) > self.max_modules:
77         self._prune(epoch)
78
79     return module_id
80
81 def update_performance(
82     self,
83     module_id: str,
84     loss: float,
85     epoch: int,
86 ) -> None:
87     """Record a module's performance on a batch."""
88     if module_id in self._modules:
89         record = self._modules[module_id]
90         record.total_loss += loss
91         record.num_uses += 1
92         record.last_used_epoch = epoch
93
94 def get_module(self, module_id: str) -> Optional[nn.Module]:
95     """Retrieve a module by ID."""
96     record = self._modules.get(module_id)
97     return record.module if record else None
98
99 def get_all_modules(self) -> list[tuple[str, nn.Module]]:
100     """Get all registered (id, module) pairs."""
101     return [(r.module_id, r.module) for r in self._modules.values()]
102
103 def get_active_modules(self) -> nn.ModuleList:
104     """Get all active modules as a ModuleList (for parameter access)."""
105     return nn.ModuleList([r.module for r in self._modules.values()])
106
107 def _prune(self, current_epoch: int) -> list[str]:
108     """Remove underperforming or stale modules.

```

```

109
110     Returns list of pruned module IDs.
111     """
112     pruned = []
113     if not self._modules:
114         return pruned
115
116     # Sort by average loss (worst first)
117     sorted_records = sorted(
118         self._modules.values(),
119         key=lambda r: r.avg_loss,
120         reverse=True,
121     )
122
123     # Remove worst performers until under capacity
124     while len(self._modules) > self.max_modules and sorted_records:
125         worst = sorted_records.pop(0)
126         # Don't prune very new modules
127         if current_epoch - worst.created_epoch < self.pruning_patience:
128             continue
129         del self._modules[worst.module_id]
130         pruned.append(worst.module_id)
131
132     # Also prune stale modules (unused for too long)
133     stale = [
134         mid for mid, r in self._modules.items()
135         if current_epoch - r.last_used_epoch > self.pruning_patience * 2
136         and r.num_uses > 0
137     ]
138     for mid in stale:
139         del self._modules[mid]
140         pruned.append(mid)
141
142     return pruned
143
144     def prune(self, current_epoch: int) -> list[str]:
145         """Public pruning interface."""
146         return self._prune(current_epoch)
147
148     @property
149     def num_modules(self) -> int:
150         return len(self._modules)
151
152     def get_statistics(self) -> dict:
153         """Summary statistics for all registered modules."""
154         if not self._modules:
155             return {"num_modules": 0, "avg_loss": 0.0, "module_types": {}}
156
157         type_counts: dict[str, int] = {}
158         total_loss = 0.0
159         total_uses = 0
160         for r in self._modules.values():
161             t = r.feature.module_type
162             type_counts[t] = type_counts.get(t, 0) + 1
163             total_loss += r.total_loss
164             total_uses += r.num_uses
165
166         return {
167             "num_modules": len(self._modules),

```

Line 64 builds ids from the module type and a monotonic counter, so logs read `mod_cycle_3` rather than a hash.

WHY CAN PRUNING LEAVE THE REGISTRY OVER CAPACITY?

Look at line 127: a module younger than `pruning_patience` is skipped rather than evicted, and the loop moves on. If every module is new, nothing gets pruned and the count stays above `max_modules` for a few epochs.

That is the intended behaviour. A module's `avg_loss` after two batches is noise. Evicting on it would systematically favour whichever modules happened to be created during an easy stretch of training, which is not a quality signal at all. Temporarily exceeding the cap is the cheaper mistake.

tcd_jepa/core/system3_crystallizer.py

lines 66-132

```
66     def crystallize(
67         self,
68         point_cloud: torch.Tensor,
69         epoch: int,
70     ) -> dict:
71         """Run full crystallization pipeline on a point cloud.
72
73         Args:
74             point_cloud: [N, D] tensor of latent points from trajectories.
75             epoch: Current training epoch.
76
77         Returns:
78             Dict with crystallization results:
79             'new_modules': list of (module_id, module_type) tuples
80             'ph_analysis': persistence diagram analysis
81             'num_features': number of topological features found
82             'total_persistence': sum of all persistence values
83         """
84         # Guard: check point cloud has sufficient points for PH
85         if point_cloud.shape[0] < 2:
86             logger.info(
87                 f"Insufficient points for crystallization ({point_cloud.shape[0]} < 2) - skipping"
88             )
89             return {
90                 "new_modules": [],
91                 "pruned_modules": [],
92                 "ph_analysis": {},
93                 "num_features": 0,
94                 "total_persistence": 0.0,
95                 "registry_stats": self.registry.get_statistics(),
96             }
97
98         # Step 1: Compute persistent homology
99         ph_result = self.ph_computer.compute(
100             point_cloud, max_points=self.max_points_for_ph
101         )
102
103         # Step 2: Analyze persistence diagrams
104         ph_analysis = self.diagram_analyzer.analyze(ph_result)
105
106         # Step 3: Extract topological features
107         features = self.feature_extractor.extract_features(
108             ph_analysis, point_cloud=point_cloud
109         )
110
111         # Step 4: Create and register modules for significant features
```

```

112     new_modules = []
113     for feature in features:
114         module = self.module_factory.create_module(feature, self.device)
115         module_id = self.registry.register(module, feature, epoch)
116         new_modules.append((module_id, feature.module_type))
117
118     # Step 5: Prune underperformers
119     pruned = self.registry.prune(epoch)
120
121     # Log
122     result = {
123         "new_modules": new_modules,
124         "pruned_modules": pruned,
125         "ph_analysis": ph_analysis,
126         "num_features": len(features),
127         "total_persistence": self.diagram_analyzer.total_persistence(ph_result),
128         "registry_stats": self.registry.get_statistics(),
129     }
130     self._crystallization_log.append(result)
131
132     return result

```

The six-stage pipeline, with a guard at line 85 for point clouds too small to filter.

Letting the modules back in

A freshly crystallized module is a randomly initialised network being inserted into a model that was converging. Almost every decision in this part is a defence against that.

Crystallized modules are useless until something calls them. `DynamicPredictor` is the return path that closes the loop — and it applies three independent brakes to the module contribution before it reaches the output.

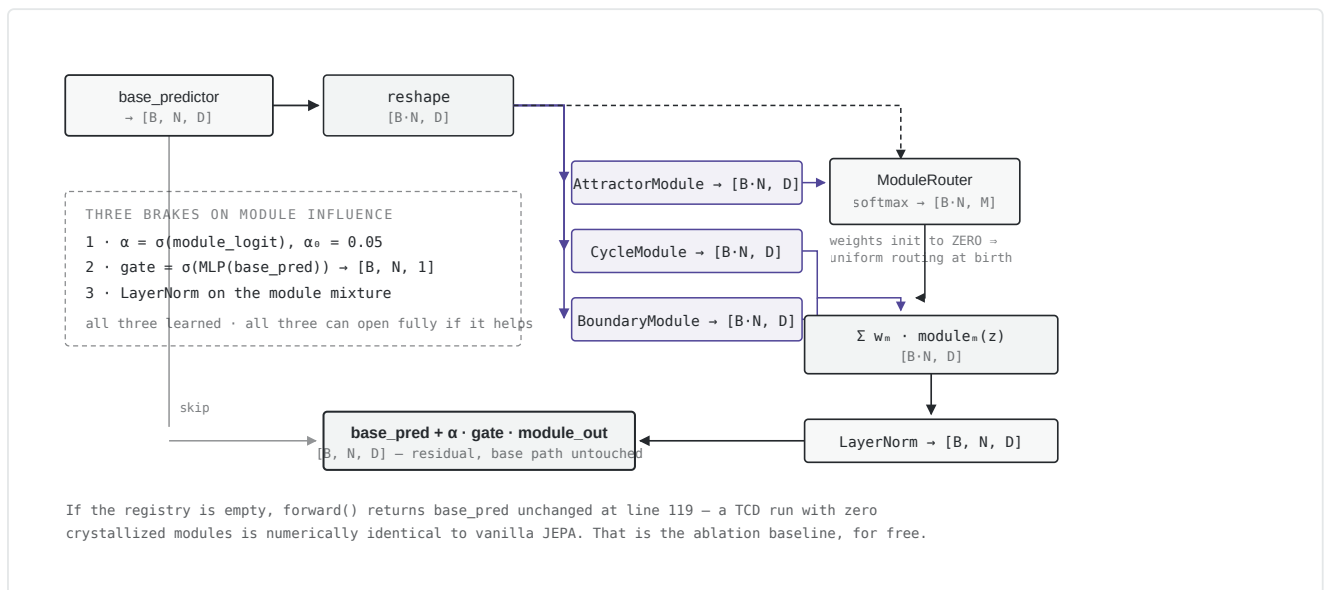


FIGURE 9 The mixing path. Three brakes in series: a learned scalar starting at 0.05, a per-token gate, and a LayerNorm on the combined module output. Only after all three is the mixture added as a residual.

WHY THREE BRAKES RATHER THAN ONE?

They control different things, and any one alone leaves a gap.

The **scalar α** , initialised at 0.05 via a sigmoid on a learned logit, controls *how much* module influence there is overall. It is the volume knob, and starting it near zero means a new module cannot damage predictions before it has trained.

The **per-token gate** controls *where* that influence applies. Modules are specialists — an attractor module is meaningful near its centroid and meaningless elsewhere. A single global scalar cannot express "relevant for these tokens, not those." The gate is a small sigmoid MLP producing one weight per token position.

The **LayerNorm** controls *magnitude*. Crystallized modules are initialised independently and nothing constrains their output scale; a module that happens to emit vectors ten times larger than the base prediction would swamp it regardless of α . Normalising the combined mixture makes α 's meaning stable — 0.05 means the same thing whatever the modules happen to output.

WHY RESIDUAL, RATHER THAN REPLACING THE BASE PREDICTION?

Because the base predictor is the thing that works. Modules are a hypothesis about extra structure; the residual form means that hypothesis can only ever be an adjustment. If every module is useless, the model can drive α to zero and it has lost nothing.

The clean consequence is at line 119: with an empty registry, `forward` returns the base prediction untouched. A TCD run that crystallizes nothing is numerically identical to vanilla JEPA — which means the ablation baseline is free and exact rather than approximate.

tcd_jepa/modules/dynamic_predictor.py

lines 20-174

```
20 class ModuleRouter(nn.Module):
21     """Learned soft router that assigns per-module weights via softmax.
22
23     Routes input representations to crystallized modules with learned
24     importance weights, enabling gradient-based module selection.
```

```

25     """
26
27     def __init__(self, embed_dim: int, max_modules: int = 64) -> None:
28         super().__init__()
29         self.embed_dim = embed_dim
30         self.max_modules = max_modules
31         self.router_proj = nn.Linear(embed_dim, max_modules)
32         nn.init.zeros_(self.router_proj.weight)
33         nn.init.zeros_(self.router_proj.bias)
34
35     def forward(self, x: torch.Tensor, num_active: int) -> torch.Tensor:
36         """Compute soft routing weights over active modules.
37
38         Args:
39             x: Input representations [B, D].
40             num_active: Number of currently active modules.
41
42         Returns:
43             Routing weights [B, num_active] summing to 1 per sample.
44         """
45         logits = self.router_proj(x)[: , :num_active] # [B, num_active]
46         return F.softmax(logits, dim=-1)
47
48
49     class DynamicPredictor(nn.Module):
50         """Predictor that combines base JEPA predictions with crystallized modules.
51
52         The base predictor handles the standard JEPA prediction task.
53         Crystallized modules contribute specialized predictions for regions
54         of latent space they were designed for.
55
56         Architecture:
57             combined = base_pred + alpha * token_gate(base_pred) * module_norm(routed_modules)
58             where alpha is a learned scalar, token_gate provides spatial selectivity,
59             and module_norm normalizes module outputs.
60         """
61
62         def __init__(
63             self,
64             base_predictor: nn.Module,
65             embed_dim: int,
66             module_weight: float = 0.05,
67             registry: Optional[ModuleRegistry] = None,
68         ) -> None:
69             super().__init__()
70             self.base_predictor = base_predictor
71             self.embed_dim = embed_dim
72             self.registry = registry or ModuleRegistry()
73
74             # Learnable mixing coefficient – start very small so new modules
75             # don't immediately corrupt predictions
76             init_logit = torch.log(torch.tensor(module_weight / (1.0 - module_weight + 1e-8)))
77             self.module_logit = nn.Parameter(init_logit)
78
79             # Learned soft router for per-module weighting
80             self.module_router = ModuleRouter(embed_dim)
81
82             # Per-token gate for spatial selectivity
83             self.token_gate = nn.Sequential(
84                 nn.Linear(embed_dim, embed_dim // 4),
85                 nn.GELU(),
86                 nn.Linear(embed_dim // 4, 1),
87                 nn.Sigmoid(),
88             )
89
90             # Output normalization for module contributions
91             self.module_norm = nn.LayerNorm(embed_dim)
92
93         @property
94         def module_weight(self) -> torch.Tensor:
95             return torch.sigmoid(self.module_logit)
96
97         def forward(
98             self,

```

```

99         x: torch.Tensor,
100         masks_x: list[torch.Tensor],
101         masks: list[torch.Tensor],
102     ) -> torch.Tensor:
103         """Forward pass combining base predictor and crystallized modules.
104
105         Args:
106             x: Encoded context patches [B*nenc, N_ctx, D].
107             masks_x: Context mask indices.
108             masks: Target mask indices.
109
110         Returns:
111             Combined predictions [B*npred*nenc, N_pred, D].
112         """
113         # Base prediction – no exception swallowing, let errors propagate
114         base_pred = self.base_predictor(x, masks_x, masks)
115
116         # If no modules registered, return base prediction
117         modules = self.registry.get_all_modules()
118         if not modules:
119             return base_pred
120
121         # Aggregate module contributions with learned routing
122         B, N, D = base_pred.shape
123         z_flat = base_pred.reshape(B * N, D)
124
125         # Compute per-token routing weights (each token gets distinct routing)
126         routing_weights = self.module_router(z_flat, len(modules)) # [B*N, M]
127
128         # Compute weighted module contributions
129         module_outputs = []
130         for module_id, module in modules:
131             try:
132                 contribution = module(z_flat) # [B*N, D]
133                 if contribution.shape != z_flat.shape:
134                     logger.warning(
135                         f"Module {module_id} output shape {contribution.shape} != "
136                         f"expected {z_flat.shape} – skipping"
137                     )
138                 continue
139                 module_outputs.append(contribution)
140             except Exception as e:
141                 logger.warning(f"Module {module_id} forward failed: {e} – skipping")
142                 continue
143
144         if module_outputs:
145             # Stack and apply routing: [B*N, M, D] * [B*N, M, 1] -> [B*N, D]
146             num_active = len(module_outputs)
147             stacked = torch.stack(module_outputs, dim=1) # [B*N, M, D]
148             # Re-compute routing for actual active modules (some may have been skipped)
149             if num_active != len(modules):
150                 routing_weights = self.module_router(z_flat, num_active)
151             weights = routing_weights[:, :num_active].unsqueeze(-1) # [B*N, M, 1]
152             module_combined = (stacked * weights).sum(dim=1) # [B*N, D]
153
154             # Normalize module output
155             module_combined = self.module_norm(module_combined)
156             module_combined = module_combined.reshape(B, N, D)
157
158             # Learned per-token gating for spatial selectivity
159             gate = self.token_gate(base_pred) # [B, N, 1]
160             alpha = self.module_weight
161             combined = base_pred + alpha * gate * module_combined
162             return combined
163
164         return base_pred
165
166     def get_module_parameters(self) -> list[nn.Parameter]:
167         """Get parameters from all registered modules (for optimizer)."""
168         params = list(self.token_gate.parameters())
169         params.extend(self.module_router.parameters())
170         params.extend(self.module_norm.parameters())
171         params.append(self.module_logit)
172         for _, module in self.registry.get_all_modules():

```



```
173     params.extend(module.parameters())
174     return params
```

The router, the gate, the norm, and the mixing.

WHY INITIALISE THE ROUTER'S WEIGHTS TO EXACTLY ZERO?

Because zero weights and zero bias make the softmax exactly uniform. Every module gets identical routing weight on the first forward pass after it is created.

The alternative — random initialisation — would hand new modules an arbitrary preference ordering that has nothing to do with their quality. The router would then have to unlearn that before it could learn anything real. Starting uniform means the first gradient the router receives is the first real information it has about which module is useful.

WHY DOES THE BASE PREDICTION HAVE NO TRY/EXCEPT WHEN THE MODULES DO?

Deliberate asymmetry, and the docstring calls it out. If the base predictor raises, the model is broken and the run should stop loudly — swallowing that would produce a training run that silently learns nothing.

A crystallized module raising is a different situation. It is one speculative component out of several, generated automatically from a topological feature, and the correct response is to skip it with a warning and carry on. Lines 149–150 then recompute the routing weights for the number of modules that actually survived, so the softmax still normalises over the right count.

WHY DO NEW MODULE PARAMETERS GET ZERO WEIGHT DECAY?

Because an attractor module's centroid and radius encode a measured location in latent space. Decaying them toward zero would erase exactly the information that justified building the module — the optimizer would slowly delete the discovery.

The same applies to the cycle module's learned frequencies. These are not generic weights where shrinkage is a sensible prior; they are parameters with meaning.

Three clocks

The three systems run at three different rates, for three different reasons — and the loop needs a way to know when it has stopped producing new structure.

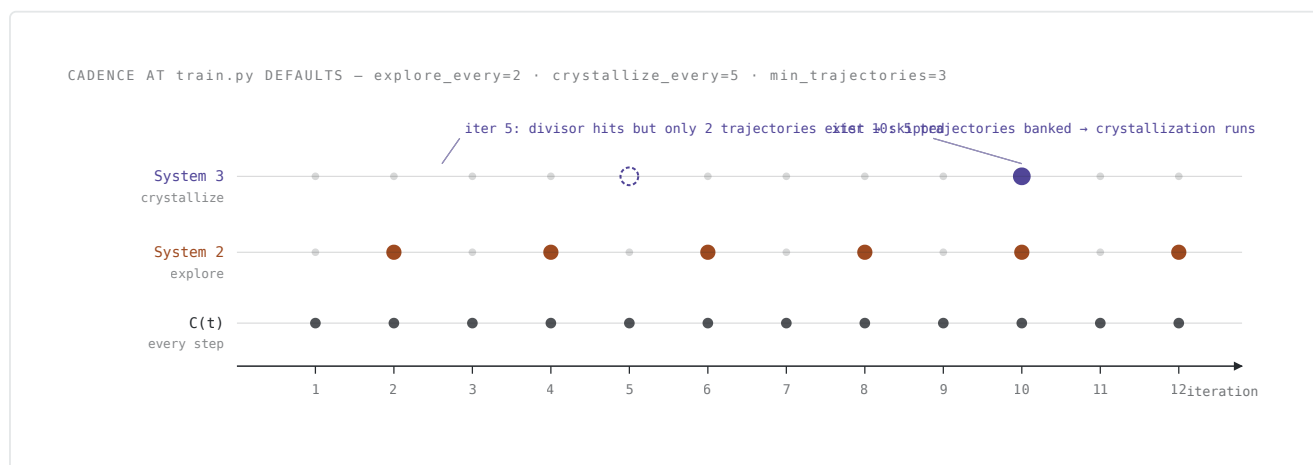


FIGURE 10 Loop cadence over twelve iterations. The dashed ring at iteration 5 is a guard doing its job: the divisor fires but not enough trajectories have accumulated, so crystallization is skipped.

WHY DOES EXPLORATION NOT RUN EVERY STEP?

Cost. One exploration cycle is 2×32 energy evaluations per point for the Hessian probe on up to 256 points, plus 20–50 Langevin steps each requiring a gradient. That is comparable to several training steps. Running it every step would roughly halve throughput for a subsystem that only needs to sample the landscape, not track it continuously.

WHY DOES CRYSTALLIZATION RUN EVEN LESS OFTEN, AND WHY THE TRAJECTORY MINIMUM?

Because persistent homology on a single trajectory is measuring noise. One walk from one set of starting points traces one path; its shape says as much about where it started as about the landscape. Several trajectories from different starting points, pooled, describe the region.

Hence two independent gates. The divisor spaces crystallization out in time; the `min_trajectories` check ensures there is enough material regardless of how the divisors happen to line up. Figure 10 shows the case where they disagree — the divisor fires at iteration 5 and the guard vetoes it.

WHY ONLY THE LAST FIVE TRAJECTORIES?

`get_point_cloud(last_n=5)` deliberately ignores older history. The energy landscape is changing as the model trains — a trajectory from thirty iterations ago describes a landscape that no longer exists. Pooling it with recent walks would blur real current structure against stale structure and depress the persistence of both.

Knowing when it is finished

```
C(t) = |M(t) - M(t-1)| / max(M(t), 1)      module for
      + KL( R(t) || R(t-1) )                representa
      + |S(t) - S(t-1)|                      energy lar

converged  $\iff C(t) < \epsilon$  for k consecutive iterations
```

WHY THREE TERMS RATHER THAN JUST COUNTING MODULES?

Because the module count alone is easy to satisfy for the wrong reason. A system that has hit `max_modules` stops creating new ones — the first term goes to zero — while the representations underneath are still churning and the landscape is still reshaping. That is not convergence; it is a full registry.

The second term catches it. Representation drift is measured as a KL divergence between histograms of latent norms across consecutive iterations, which detects the encoder still moving even when the architecture has stopped changing. The third term watches the energy landscape's ruggedness. Convergence means all three have settled: the architecture is stable, the representations are stable, and the landscape they define is stable.

tcd_jepa/core/recursive_loop.py

lines 27–141

```
27 class ConvergenceMonitor:
28     """Tracks convergence of the recursive loop.
29
30     C(t) = module_change + representation_divergence + smoothness_change
31
32     Converged when C(t) < epsilon for k consecutive iterations.
33     """
34
35     def __init__(
36         self,
37         epsilon: float = 0.01,
38         patience: int = 5,
39         history_size: int = 50,
40         distributed: bool = False,
41     ) -> None:
42         self.epsilon = epsilon
43         self.patience = patience
44         self.distributed = distributed
45         self._history: list[dict] = []
46         self._convergence_scores: list[float] = []
47         self._consecutive_converged = 0
48         self._prev_num_modules = 0
49         self._prev_repr_histogram: Optional[torch.Tensor] = None
50         self._prev_smoothness = 0.0
51
52     def update(
53         self,
54         num_modules: int,
55         representations: torch.Tensor,
56         energy_landscape_smoothness: float,
57     ) -> dict:
58         """Record one iteration and compute convergence metrics.
59
60         Args:
61             num_modules: Current active module count M(t).
62             representations: Current encoder outputs [B, D] for distribution tracking.
63             energy_landscape_smoothness: Energy std as smoothness proxy.
64
65         Returns:
66             Dict with convergence metrics and is_converged flag.
```

```

67     """
68     # Module formation rate:  $|M(t) - M(t-1)| / \max(M(t), 1)$ 
69     module_change = abs(num_modules - self._prev_num_modules) / max(num_modules, 1)
70
71     # Representation distribution shift (histogram-based KL approximation)
72     repr_histogram = self._compute_repr_histogram(representations)
73     if self._prev_repr_histogram is not None:
74         repr_divergence = self._kl_divergence(repr_histogram, self._prev_repr_histogram)
75     else:
76         repr_divergence = 1.0
77
78     # Energy landscape smoothness change
79     smoothness_change = abs(energy_landscape_smoothness - self._prev_smoothness)
80
81     # Combined convergence score
82     convergence_score = module_change + repr_divergence + smoothness_change
83
84     # Synchronize across ranks: use mean convergence score
85     if self.distributed and dist.is_initialized():
86         score_tensor = torch.tensor([convergence_score], dtype=torch.float64)
87         dist.all_reduce(score_tensor, op=dist.ReduceOp.AVG)
88         convergence_score = float(score_tensor.item())
89
90     # Track consecutive convergence
91     if convergence_score < self.epsilon:
92         self._consecutive_converged += 1
93     else:
94         self._consecutive_converged = 0
95
96     is_converged = self._consecutive_converged >= self.patience
97
98     # Update state
99     self._prev_num_modules = num_modules
100    self._prev_repr_histogram = repr_histogram
101    self._prev_smoothness = energy_landscape_smoothness
102
103    result = {
104        "convergence_score": convergence_score,
105        "module_change": module_change,
106        "repr_divergence": repr_divergence,
107        "smoothness_change": smoothness_change,
108        "is_converged": is_converged,
109        "consecutive_converged": self._consecutive_converged,
110    }
111    self._convergence_scores.append(convergence_score)
112    self._history.append(result)
113    return result
114
115    @staticmethod
116    def _compute_repr_histogram(
117        representations: torch.Tensor,
118        num_bins: int = 50,
119    ) -> torch.Tensor:
120        """Compute a normalized histogram of representation norms."""
121        norms = representations.detach().norm(dim=-1).cpu()
122        hist = torch.histc(norms, bins=num_bins, min=0.0, max=float(norms.max()) + 1e-6)
123        return hist / (hist.sum() + 1e-8)
124
125    @staticmethod
126    def _kl_divergence(p: torch.Tensor, q: torch.Tensor) -> float:
127        """Compute KL(p || q) with smoothing."""
128        p = p + 1e-8
129        q = q + 1e-8
130        p = p / p.sum()
131        q = q / q.sum()
132        return float(F.kl_div(q.log(), p, reduction="sum").item())
133
134    @property
135    def history(self) -> list[dict]:
136        return self._history
137
138    @property
139    def is_converged(self) -> bool:
140        return self._consecutive_converged >= self.patience

```

The convergence monitor.

WHY IS THE FIRST ITERATION'S DIVERGENCE HARD-CODED TO 1.0?

Line 76. On the very first call there is no previous histogram to compare against, and the honest answer is undefined. Setting it to zero would be the natural-looking choice and would be a bug: with zero divergence and zero module change, a cold system could satisfy $C(t) < \epsilon$ on its first few iterations and declare convergence before doing anything.

Forcing 1.0 guarantees the first score is far above ϵ . The cost is that the first entry in the convergence history is not comparable with the rest, which is why the analysis scripts drop it.

WHY ALL-REDUCE THE CONVERGENCE SCORE UNDER DDP?

Lines 85–88. Each rank computes its score on its own data shard, so scores differ. Without synchronisation, rank 0 could decide the system had converged while rank 3 disagreed — and since convergence gates behaviour, the ranks would diverge in what they did next. Averaging makes it a collective decision.

tcd_jepa/core/recursive_loop.py

lines 194–270

```

194     def step(
195         self,
196         z_context: torch.Tensor,
197         energy_fn: callable,
198         predictor_fn: Optional[callable] = None,
199         epoch: int = 0,
200     ) -> dict:
201         """Execute one iteration of the recursive loop.
202
203         Args:
204             z_context: Current encoder output [B, N, D] or [B, D].
205             energy_fn: Energy function mapping latent points to scalars.
206             predictor_fn: Optional predictor for variance-based detection.
207             epoch: Current training epoch.
208
209         Returns:
210             Dict with exploration results, crystallization results, convergence.
211         """
212         self._iteration += 1
213         result = {
214             "iteration": self._iteration,
215             "explored": False,

```

```

216         "crystallized": False,
217     }
218
219     # System 2: Explore (periodic)
220     if self._iteration % self.explore_every == 0:
221         explore_result = self.explorer.explore(
222             z_context, energy_fn, predictor_fn, epoch=epoch
223         )
224         result["exploration"] = {
225             "num_blank": int(explore_result["blank_info"]["is_blank"].sum()),
226             "mean_fisher_trace": float(explore_result["fisher_trace"].mean()),
227             "trajectory_length": explore_result["trajectory"].shape[0],
228         }
229         result["explored"] = True
230
231     # System 3: Crystallize (less frequent, needs enough trajectories)
232     if (self._iteration % self.crystallize_every == 0
233         and self.explorer.num_trajectories >= self.min_trajectories):
234         try:
235             point_cloud = self.explorer.get_point_cloud(last_n=5)
236         except ValueError:
237             point_cloud = None
238
239         if point_cloud is not None and point_cloud.shape[0] >= 2:
240             crystal_result = self.crystallizer.crystallize(point_cloud, epoch)
241             result["crystallization"] = {
242                 "new_modules": crystal_result["new_modules"],
243                 "pruned_modules": crystal_result["pruned_modules"],
244                 "total_persistence": crystal_result["total_persistence"],
245                 "num_active_modules": self.crystallizer.num_modules,
246             }
247             # Expose new module objects – filter out None modules
248             result["new_module_objects"] = [
249                 (mid, self.crystallizer.registry.get_module(mid), feat)
250                 for mid, feat in crystal_result["new_modules"]
251                 if self.crystallizer.registry.get_module(mid) is not None
252             ]
253             result["crystallized"] = True
254
255     # Convergence monitoring
256     if z_context.dim() == 3:
257         z_flat = z_context.reshape(-1, z_context.shape[-1])
258     else:
259         z_flat = z_context
260
261     smoothness = float(energy_fn(z_flat[:min(64, len(z_flat))]).std())
262
263     conv = self.convergence.update(
264         num_modules=self.crystallizer.num_modules,
265         representations=z_flat[:min(256, len(z_flat))],
266         energy_landscape_smoothness=smoothness,
267     )
268     result["convergence"] = conv
269
270     return result

```

The step function. Line 261 computes the landscape-smoothness proxy: the standard deviation of energy across up to 64 latent points.

Keeping it alive

One training step has four points where it can refuse to do the normal thing.

Every one exists because of the asymmetry established in Part II: the EMA target encoder is the component gradient descent cannot repair.

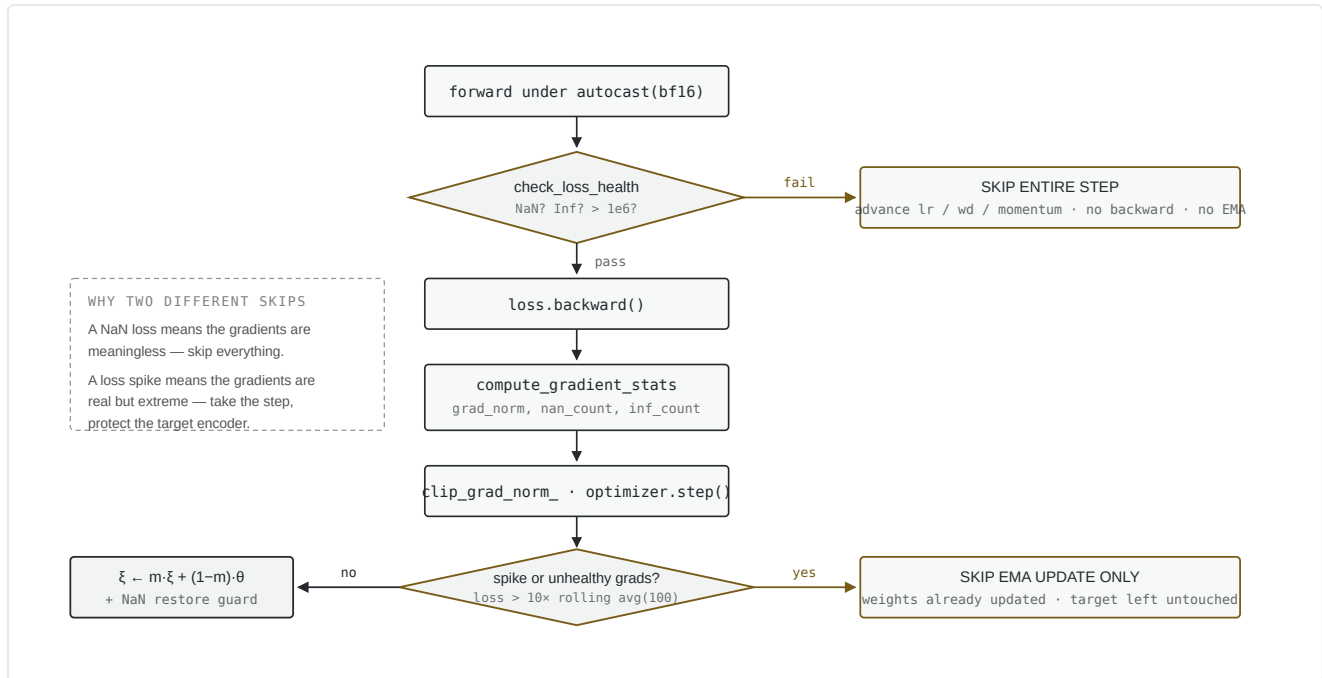


FIGURE 11 One call to `Trainer._train_step`. Two different failure responses, deliberately distinguished.

WHY DO A NaN LOSS AND A LOSS SPIKE GET DIFFERENT TREATMENT?

Because they mean different things.

A **NaN or infinite loss** means the gradients derived from it are meaningless. There is nothing to be learned from this batch, so the whole step is abandoned: no backward pass, no optimizer step, no EMA update.

A **loss spike** — more than ten times the hundred-step rolling average — means something quite different. The gradients are real; the batch was just hard. Throwing away a real gradient because it was large is wasteful, and gradient clipping already bounds how far the parameters can move. So the optimizer step proceeds and only the EMA update is skipped.

The reason to protect specifically the EMA is that its update is not clipped and not reversible. A single extreme step pulls the target encoder somewhere it will take hundreds of steps to drift back from — and every target produced in the meantime is degraded.

WHY DO THE LR, WEIGHT-DECAY AND MOMENTUM SCHEDULES ADVANCE EVEN ON A SKIPPED STEP?

Because they are indexed against a total step count computed once at startup: `num_epochs × steps_per_epoch`. If a skipped step also skipped the schedules, they would finish late — the cosine decay would not reach its endpoint, and the momentum schedule would never reach 1.0. Advancing them keeps the schedules synchronised with wall-clock training regardless of how many steps were usable.

tcd_jepa/training/trainer.py

lines 187–262

```
187     def _train_step(  
188         self,  
189         images: torch.Tensor,  
190         masks_enc: list[torch.Tensor],  
191         masks_pred: list[torch.Tensor],  
192     ) -> dict:  
193         """Execute a single training step.  
194
```

```

195     Returns:
196         Dict with loss, lr, wd, momentum, grad stats, and skip indicators.
197     """
198     # Move data to device
199     images = images.to(self.device)
200     masks_enc = [m.to(self.device) for m in masks_enc]
201     masks_pred = [m.to(self.device) for m in masks_pred]
202
203     # Forward pass with optional mixed-precision autocast
204     with torch.amp.autocast("cuda", dtype=torch.bfloat16, enabled=self.use_amp):
205         result = self.model(images, masks_enc, masks_pred)
206         loss = result["loss"]
207
208     # Check loss health before backward
209     loss_healthy = check_loss_health(loss)
210     if not loss_healthy:
211         # Skip this step entirely – advance schedules but don't update weights/EMA
212         new_lr = self.lr_scheduler.step()
213         new_wd = self.wd_scheduler.step()
214         new_m = next(self.momentum_schedule)
215         return {
216             "loss": 0.0, "lr": new_lr, "wd": new_wd, "momentum": new_m,
217             "grad_norm": 0.0, "param_norm": 0.0,
218             "nan_count": 1, "inf_count": 0, "is_spike": False, "skipped": True,
219         }
220
221     # Backward pass with gradient clipping
222     self.optimizer.zero_grad()
223     if self.scaler is not None:
224         self.scaler.scale(loss).backward()
225         self.scaler.unscale_(self.optimizer)
226         grad_stats = compute_gradient_stats(self.model)
227         torch.nn.utils.clip_grad_norm_(self.model.parameters(), self.grad_clip_norm)
228         self.scaler.step(self.optimizer)
229         self.scaler.update()
230     else:
231         loss.backward()
232         grad_stats = compute_gradient_stats(self.model)
233         torch.nn.utils.clip_grad_norm_(self.model.parameters(), self.grad_clip_norm)
234         self.optimizer.step()
235
236     # Update schedules
237     new_lr = self.lr_scheduler.step()
238     new_wd = self.wd_scheduler.step()
239     new_m = next(self.momentum_schedule)
240
241     # Check for loss spike and gradient health before EMA update
242     loss_val = loss.item()
243     is_spike = self._loss_tracker.update(loss_val)
244     skip_ema = is_spike or not grad_stats.is_healthy
245
246     if not skip_ema:
247         self.model.update_target_encoder(new_m)
248     else:
249         logger.warning("Skipping EMA update due to unhealthy step")
250
251     return {
252         "loss": loss_val,
253         "lr": new_lr,
254         "wd": new_wd,
255         "momentum": new_m,
256         "grad_norm": grad_stats.grad_norm,
257         "param_norm": grad_stats.param_norm,
258         "nan_count": grad_stats.nan_count,
259         "inf_count": grad_stats.inf_count,
260         "is_spike": is_spike,
261         "skipped": False,
262     }

```

Line 244: skip_ema = is_spike or not grad_stats.is_healthy .

```

33 def check_loss_health(loss: torch.Tensor, threshold: float = 1e6) -> bool:
34     """Check if loss is finite and within reasonable bounds.
35
36     Returns:
37         True if loss is healthy, False if NaN/Inf/exceeds threshold.
38     """
39     if torch.isnan(loss):
40         logger.warning("NaN loss detected – skipping step")
41         return False
42     if torch.isinf(loss):
43         logger.warning("Inf loss detected – skipping step")
44         return False
45     if loss.item() > threshold:
46         logger.warning(f"Loss exceeds threshold ({loss.item():.2e} > {threshold:.2e}) – skipping step")
47         return False
48     return True
49
50
51 def compute_gradient_stats(model: nn.Module) -> GradientStats:
52     """Compute gradient health statistics for all model parameters.
53
54     Call after backward() and before optimizer.step() to monitor gradient health.
55     """
56     stats = GradientStats()
57     grad_norms_sq = 0.0
58     param_norms_sq = 0.0
59     total_params = 0
60
61     for p in model.parameters():
62         if p.grad is not None:
63             g = p.grad.data
64             grad_norms_sq += g.norm(2).item() ** 2
65             g_abs_max = g.abs().max().item()
66             g_abs_min = g.abs().min().item()
67             stats.grad_max = max(stats.grad_max, g_abs_max)
68             stats.grad_min = min(stats.grad_min, g_abs_min)
69             stats.nan_count += torch.isnan(g).sum().item()
70             stats.inf_count += torch.isinf(g).sum().item()
71             total_params += 1
72
73             param_norms_sq += p.data.norm(2).item() ** 2
74
75     stats.grad_norm = grad_norms_sq**0.5
76     stats.param_norm = param_norms_sq**0.5
77
78     if total_params == 0:
79         stats.grad_min = 0.0
80
81     if stats.nan_count > 0:
82         logger.warning(f"NaN gradients detected: {stats.nan_count} elements")
83     if stats.inf_count > 0:
84         logger.warning(f"Inf gradients detected: {stats.inf_count} elements")
85
86     return stats
87
88
89 class LossTracker:
90     """Tracks loss values over a rolling window to detect spikes.
91
92     A spike is detected when the current loss exceeds `spike_factor` times
93     the rolling average. When a spike is detected, EMA updates should be skipped
94     to prevent corrupting the target encoder.
95     """
96
97     def __init__(self, window_size: int = 100, spike_factor: float = 10.0) -> None:
98         self.window: deque[float] = deque(maxlen=window_size)
99         self.spike_factor = spike_factor
100         self.total_spikes = 0
101
102     @property
103     def rolling_avg(self) -> float:

```

```

104         if not self.window:
105             return 0.0
106         return sum(self.window) / len(self.window)
107
108     def update(self, loss: float) -> bool:
109         """Record a loss value and check for spikes.
110
111         Returns:
112             True if loss is a spike (should skip EMA update).
113         """
114         avg = self.rolling_avg
115         is_spike = False
116
117         if len(self.window) >= 10 and avg > 0:
118             if loss > self.spike_factor * avg:
119                 logger.warning(
120                     f"Loss spike detected: {loss:.4f} > {self.spike_factor}x rolling avg {avg:.4f}"
121                 )
122                 is_spike = True
123                 self.total_spikes += 1
124
125         self.window.append(loss)
126         return is_spike

```

`LossTracker` needs at least ten samples before it will call anything a spike, so the noisy first steps of training cannot trigger it.

Where the loop attaches

WHY DOES THE RECURSIVE LOOP RUN ONCE PER EPOCH RATHER THAN PER STEP?

Partly cost, as in Part VIII. But mostly because the thing it measures does not change on a per-step timescale. The energy landscape is defined by the encoder and predictor; after one optimizer step it is essentially the landscape from before. Exploring it again would produce trajectories nearly identical to the last set, and pooling near-duplicates inflates persistence without adding information.

An epoch is a natural interval over which the landscape has genuinely moved. The energy function is rebuilt from the current target encoder each time, so System 2 always explores the present landscape rather than a cached one.

`tcd_jepa/training/trainer.py`

lines 100-121

```

100     # Run recursive loop at end of epoch if enabled
101     if self.recursive_loop is not None and self.stream_encoder is not None:
102         with torch.no_grad():
103             z = self.model.context_encoder(images.to(self.device))
104             t = self.model.target_encoder(images.to(self.device))
105             energy_fn = self.stream_encoder.make_energy_fn(t)
106             loop_result = self.recursive_loop.step(z, energy_fn, epoch=epoch)
107             if loop_result.get("crystallized"):

```

```

108         n_mods = loop_result["crystallization"]["num_active_modules"]
109         logger.info(f" Recursive loop: {n_mods} active modules")
110         # Add new module parameters to optimizer
111         self._register_new_module_params()
112
113         epoch_time = time.time() - t0
114         metrics_dict = epoch_metrics.to_dict()
115         metrics_dict["epoch"] = epoch
116         metrics_dict["epoch_time"] = epoch_time
117
118         # Add recursive loop metrics
119         if self.recursive_loop is not None:
120             metrics_dict["num_modules"] = self.recursive_loop.num_modules
121             metrics_dict["converged"] = self.recursive_loop.is_converged

```

The epoch-boundary hook.

tcd_jepa/training/trainer.py

lines 163-185

```

163     def _register_new_module_params(self) -> None:
164         """Add newly crystallized module parameters to the optimizer."""
165         if self.recursive_loop is None:
166             return
167         registry = self.recursive_loop.crystallizer.registry
168         new_params = []
169         for module_id, module in registry.get_all_modules():
170             if module_id not in self._known_module_ids:
171                 self._known_module_ids.add(module_id)
172                 params = list(module.parameters())
173                 if params:
174                     new_params.extend(params)
175                     module.to(self.device)
176
177         if new_params:
178             # Add as a new param group with reduced learning rate
179             current_lr = self.optimizer.param_groups[0]["lr"]
180             self.optimizer.add_param_group({
181                 "params": new_params,
182                 "lr": current_lr,
183                 "weight_decay": 0.0,
184             })
185             logger.info(f" Added {len(new_params)} new module params to optimizer")

```

New modules did not exist when the optimizer was built, so they have to be introduced.

WHY TRACK WHICH MODULE IDS HAVE ALREADY BEEN SEEN?

Because `add_param_group` does not deduplicate. Adding the same parameter twice gives it two optimizer states and effectively doubles its learning rate — a subtle bug that would look like instability confined to crystallized modules. The `_known_module_ids` set makes registration idempotent.

What --tcd actually changes

train.py

lines 287–302

```
287     # Optional TCD recursive loop
288     recursive_loop = None
289     stream_encoder = None
290     if args.tcd:
291         recursive_loop = RecursiveLoop(
292             embed_dim=enc_cfg["embed_dim"],
293             explore_every=2,
294             crystallize_every=5,
295             langevin_steps=20,
296             device=device,
297         )
298     stream_encoder = StreamEncoder(model.context_encoder, model.target_encoder)
299     # Share crystallizer's registry with the dynamic predictor
300     model.set_module_registry(recursive_loop.crystallizer.registry)
301     logger.info("TCD recursive loop enabled")
302
```

Sixteen lines is the entire difference between a vanilla run and a self-organizing one. Line 300 is the wiring that closes the loop.

WHY IS THE REGISTRY SHARED RATHER THAN COPIED?

This one line is what makes the system recursive instead of a one-way pipeline. System 3 writes modules into the registry; the predictor reads modules out of it. They are the same object, so a module registered during crystallization is live in the very next forward pass.

Copy it and you get a pipeline: the crystallizer discovers structure and nothing ever uses it. The whole feedback loop in Figure 1 runs through this single shared reference.

EXHIBIT · WHAT THE FLAG SWITCHES ON

COMPONENT	VANILLA RUN	WITH <code>--TCD</code>
Predictor	<code>VisionTransformerPredictor</code>	wrapped in <code>DynamicPredictor</code>
Recursive loop	<code>none</code>	<code>explore_every=2</code> , <code>crystallize_every=5</code> , <code>langevin_steps=20</code>
Stream encoder	<code>none</code>	constructed over context + target encoders
Registry wiring	<code>none</code>	<code>set_module_registry(loop.crystallizer.registry)</code>
Loss	<code>jepa_loss</code>	<code>jepa_loss</code> — unchanged in the default path

COMPONENT	VANILLA RUN	WITH <code>--TCD</code>
Optimizer groups	2 (decay / no-decay)	2 + one per crystallization event

The same machine on graphs

Graph mode required a new encoder, a new masking scheme and a new dataset — and not one line of change in the exploration or crystallization code. That is the payoff for the interface decision in Part IV.

In graph mode a "token" is an entity sitting at a point on a sphere. Four signals compose into its embedding: a 384-dimensional sentence-transformer fingerprint, an S^2 position, a tangent-space velocity, and eight structural features summarising its causal neighbourhood.

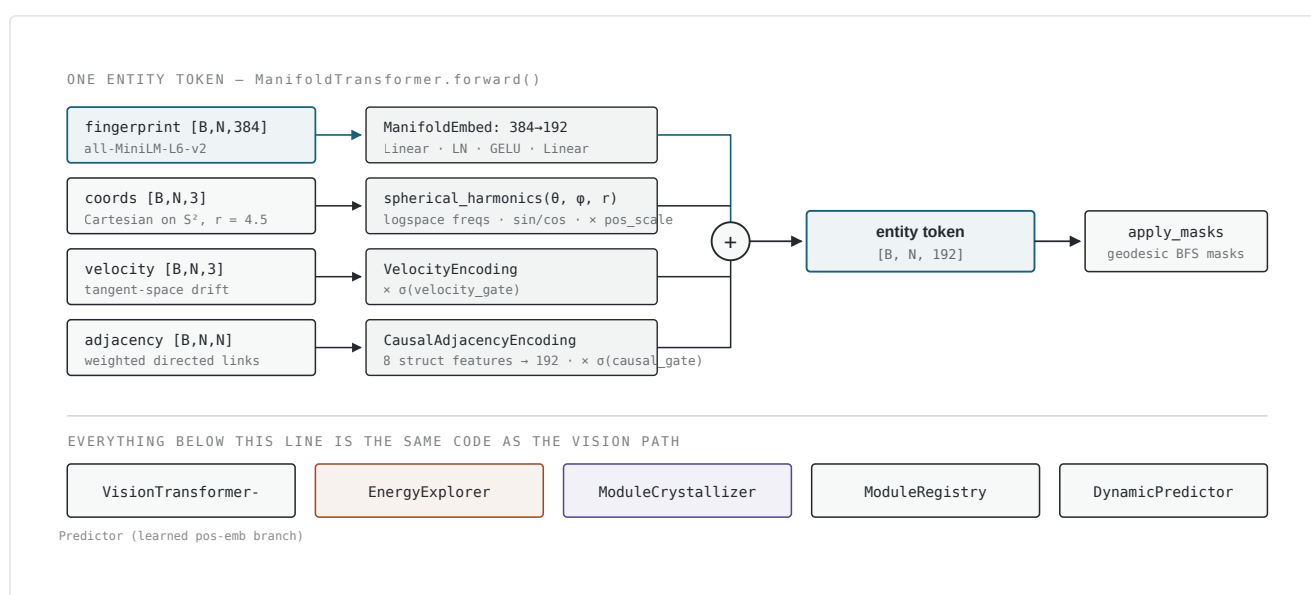


FIGURE 12 Manifold token construction. Everything below the line is byte-identical to the vision path — the same explorer, the same crystallizer, the same registry, the same predictor.

WHY DOES THE IDENTICAL PIPELINE TRANSFER AT ALL?

Because of what System 2 and System 3 actually consume. System 2 takes a callable and a tensor of points. System 3 takes an `[N, D]` array. Neither has any notion of what the dimensions mean.

Persistent homology on a cloud of latents does not care whether those latents came from image patches or from companies in a supply chain. The geometry is the geometry. That is not a lucky coincidence — it is why the interface was defined as a bare callable in the first place, and Part IV flagged this as the decision graph mode would depend on.

WHY SPHERICAL HARMONICS INSTEAD OF THE 2-D SINCOS GRID?

Because entities live on a sphere and a 2-D grid encoding has no meaning there. The vision positional encoding assumes a rectangular lattice where "one step right" is well-defined everywhere. On S^2 it is not, and using the grid encoding would give nearby points on the sphere arbitrarily different codes near the poles and the date line.

The manifold encoding converts to (θ, ϕ, r) and applies a log-spaced multi-frequency sin/cos basis, so nearby points on the sphere receive similar codes and antipodal points maximally different ones — the property the grid encoding provides on a plane, reconstructed for a sphere.

tcd_jepa/manifold/encoder.py

lines 321-381

```
321     def forward(
322         self,
323         x: torch.Tensor,
324         masks: Optional[list[torch.Tensor]] = None,
325         coords: Optional[torch.Tensor] = None,
326         adjacency: Optional[torch.Tensor] = None,
327         velocity: Optional[torch.Tensor] = None,
328         causal_features: Optional[torch.Tensor] = None,
329     ) -> torch.Tensor:
330         """Encode manifold entity tokens.
331
332     Args:
333         x: Fingerprint vectors [B, N, 384].
334         masks: Optional index tensors for token selection.
335         coords: S2 positions [B, N, 3] (Cartesian, radius ~4.5).
```

```

336         adjacency: Weighted directed adjacency [B, N, N].
337         velocity: Tangent-space velocities [B, N, 3].
338         causal_features: Pre-computed causal features [B, N, 8].
339
340     Returns:
341         Encoded representations [B', N', D].
342     """
343     if masks is not None and not isinstance(masks, list):
344         masks = [masks]
345
346     # Project fingerprints
347     x = self.patch_embed(x)
348     B, N, D = x.shape
349
350     # Add spherical positional encoding
351     if coords is not None:
352         pos = spherical_harmonics_encoding(coords, D, sphere_radius=self.sphere_radius)
353         x = x + self.pos_scale * self.pos_proj(pos)
354
355     # Add velocity encoding (temporal dynamics)
356     if self.use_velocity_encoding and velocity is not None:
357         vel_enc = self.velocity_encoding(velocity, coords)
358         gate = torch.sigmoid(self.velocity_gate)
359         x = x + gate * vel_enc
360
361     # Add causal adjacency encoding
362     if self.use_causal_encoding and (adjacency is not None or causal_features is not None):
363         if adjacency is not None:
364             causal_enc = self.causal_encoding(adjacency, causal_features)
365         else:
366             causal_enc = self.causal_encoding(
367                 torch.zeros(B, N, N, device=x.device), causal_features
368             )
369         gate = torch.sigmoid(self.causal_gate)
370         x = x + gate * causal_enc
371
372     # Apply masks (same mechanism as ViT – select subset of tokens)
373     if masks is not None:
374         x = apply_masks(x, masks)
375
376     # Transformer blocks
377     for blk in self.blocks:
378         x = blk(x)
379
380     x = self.norm(x)
381     return x

```

Structurally parallel to the ViT forward in Part II: compose the token, add positional information, apply masks, run the blocks, norm.

WHY GATE THE VELOCITY AND CAUSAL ENCODINGS?

Both enter through a learned sigmoid gate. Not every graph has meaningful velocity, and not every graph has meaningful causal structure — a static co-occurrence graph has neither. Adding an uninformative signal unconditionally injects noise into every token.

The gate lets the model shut a channel it cannot use. Since it is learned, the decision is made per-dataset by training rather than per-dataset by a human reading a config file.

```

19
20
21 def great_circle_distance(coords1: torch.Tensor, coords2: torch.Tensor, radius: float = 4.5) ->
    torch.Tensor:
22     """Compute great-circle distance between points on  $S^2$ .
23
24     Args:
25         coords1: [N, 3] Cartesian coordinates.
26         coords2: [M, 3] Cartesian coordinates.
27         radius: Sphere radius (4.5 in Latent Ocean).
28
29     Returns:
30         [N, M] pairwise great-circle distances.
31     """
32     # Normalize to unit sphere
33     c1 = coords1 / (coords1.norm(dim=-1, keepdim=True) + 1e-8)
34     c2 = coords2 / (coords2.norm(dim=-1, keepdim=True) + 1e-8)
35
36     # Cosine of angular distance
37     cos_angle = torch.mm(c1, c2.t()).clamp(-1, 1)
38
39     # Great-circle distance = R * arccos(dot product)
40     return radius * torch.acos(cos_angle)

```

Great-circle distance replaces rectangular block geometry.

WHY GROW TARGET MASKS BY BFS ON THE CAUSAL GRAPH?

Because this is the graph translation of the block-masking argument from Part III. There, the point of a contiguous block was that it removes local context and forces the model to reason from elsewhere. On a graph, "contiguous" means connected — so the target is a connected subgraph grown by breadth-first search from a seed node.

Masking random scattered nodes would leave every hidden node surrounded by visible neighbours, and the model could interpolate from immediate neighbourhood structure. Masking a connected component forces prediction across a real gap.

```

50     # Positional embeddings for predictor
51     # Use 2D sincos for perfect-square token counts (vision), learnable otherwise (manifold)
52     grid_size = int(num_patches**0.5)
53     if grid_size * grid_size == num_patches:
54         self.predictor_pos_embed = nn.Parameter(
55             torch.zeros(1, num_patches, predictor_embed_dim), requires_grad=False
56         )
57     predictor_pos_embed = get_2d_sincos_pos_embed(
58         predictor_embed_dim, grid_size, cls_token=False
59     )
60     self.predictor_pos_embed.data.copy_(
61         torch.from_numpy(predictor_pos_embed).float().unsqueeze(0)
62     )

```

```

63         else:
64             # Learnable positional embeddings for non-grid token layouts (manifold data)
65             self.predictor_pos_embed = nn.Parameter(
66                 torch.zeros(1, num_patches, predictor_embed_dim)
67             )
68             nn.init.trunc_normal_(self.predictor_pos_embed, std=0.02)

```

The one concession the shared predictor makes to graph mode.

WHY DOES THE PREDICTOR BRANCH ON WHETHER THE TOKEN COUNT IS A PERFECT SQUARE?

Because 2-D sincos positional embeddings require a square grid to exist. 64 tokens is 8×8 , so the manifold default still gets the frozen sinusoidal embeddings; an arbitrary entity count has no grid interpretation and falls back to learnable embeddings.

This is the only place in the shared code where graph mode is visible at all — everything else was absorbed by writing new encoder, masking and dataset modules.

EXHIBIT • DATA ADAPTERS

ADAPTER	DOMAIN	SCALE
eto_semiconductor.py	CSET semiconductor supply chain	519 entities
gdelt_events.py	Global news event graph	380 entities
sec_edgar.py	SEC filings co-reference graph	9,725 / ~3.9M edges
ogbn_arxiv.py · wikidata5m.py	Citation and knowledge graphs	—
uspto_patents.py · pubmed_kg.py	Patent and biomedical graphs	—
commoncrawl_webgraph.py · supply_graph.py	Web hyperlink, generic supply chains	—

Each converts a public dataset into the manifold format — fingerprints, coordinates, velocity, adjacency, optional labels. `scripts/gnn_baselines.py` runs GAT, GCN and GraphSAGE on identical splits, which is what makes the comparison in Part XI meaningful.

What the evidence says

The repository ships its own contradictory results. Graph benchmarks are strong; the headline image result does not replicate across seeds. Both are recorded here as the repository records them.

EXHIBIT · SELF-SUPERVISED LINK PREDICTION, AUC

MODEL	CSET SEMICONDUCTOR 519 ENTITIES	GDELT EVENTS 380 ENTITIES	SEC EDGAR 9,725 / ~3.9M EDGES
Crystara	82.7%	69.1%	66.4%
GAT (DeepMind)	70.3%	92.1%	out of memory
GCN (Google Brain)	63.9%	85.2%	90.8% edges / ~7% cls
GraphSAGE	33.8%	59.9%	no usable output
Baseline JEPA	46.1%	47.0%	46.4%
Δ vs baseline JEPA	+36.6 pts	+22.1 pts	+20.0 pts

The pattern the repository draws from this is that the method is stronger where the graph has physical-cluster structure and weaker where it does not. Semiconductor supply chains are genuinely organised into clusters and pipelines; news-event co-occurrence graphs are not, and GAT and GCN beat Crystara there by a wide margin.

SEC EDGAR is a scaling result rather than an accuracy one. Crystara is the only model that finishes and produces a usable downstream classification number — but 66.4% is not benchmark-leading in absolute terms, and no supervised baseline both scales and classifies well enough to compare against.

Quoting `results/GRAPH_BENCHMARKS.md` directly: *"The headline '+20 to +36.6 AUC pts over baseline JEPA' is accurate; 'beats every GNN everywhere' would not be."*

The qualitative result

On the CSET semiconductor graph the pipeline crystallized sixteen modules that map one-to-one onto real supply-chain clusters, discovered from persistent homology on Langevin trajectories with no labels and no prompting. Nine are named in the artifact.

EXHIBIT · NAMED MODULES, RESULTS/GRAPH_BENCHMARKS/SEMICONDUCTOR_MODULES.JSON

CRYSTALLIZED MODULE	CHARACTER
CMP pipeline	process-stage cluster
Netherlands / ASML lithography	national equipment ecosystem
Singapore ATP corridor	assembly-test-packaging geography
China packaging cluster	national packaging geography
Design-to-fab chain	vertical pipeline
Specialty chemicals cluster	materials supply
EUV ↔ etch/clean flows	hidden dependency
AI ASICs ↔ Hitachi dependency	hidden dependency
Lithography ↔ CMP handoff	stage interface

The last two named modules are the interesting ones. They are dependencies that do not appear as obvious clusters — which is what a topological method should find that a clustering method would not, and is the concrete version of the argument in Part VI about why H_1 and H_2 are worth the cost.

The image results, and why they do not hold up

EXHIBIT · FINAL TRAINING LOSS, FROM RESULTS/ANALYSIS.MD

SETTING	VANILLA	TCD-JEPA	IMPROVEMENT	SEEDS
CIFAR-10 ablation · dim 192, depth 6	0.005384	0.002111	+60.8%	1
Multi-modal CIFAR · dim 192, depth 6	0.005384	0.004565	+15.2%	1
Two Rooms · dim 192, depth 6	0.003846	0.004032	−4.8%	1
Synthetic images · dim 64, depth 2	0.1097 ± 0.0090	0.1102 ± 0.0084	−0.5% ± 0.6%	2
Two Rooms · dim 64, depth 2	0.0839 ± 0.0011	0.0831 ± 0.0008	+0.9% ± 0.4%	2

The 60.8% figure was a single seed at `embed_dim=192, depth=6`. Re-running at `embed_dim=64, depth=2` across two seeds produced essentially nothing. The analysis attributes the gap to capacity: at the smaller size the crystallized modules have no room to contribute.

What *does* replicate at every scale tested is module formation itself. Four modules formed reliably in both experiments across both seeds, and convergence arrived within one to

three epochs of the vanilla run. The mechanism works; its benefit on small image models does not survive a seed sweep.

HOW MUCH WEIGHT SHOULD THESE NUMBERS CARRY?

Not much, in either direction. The image results are two-seed comparisons at a scale the repository's own analysis calls too small. The graph results are single runs per benchmark with no variance estimate at all.

Read the graph deltas as promising and the 60.8% as a single-seed artifact. The strongest evidence in the repository is not any of these numbers — it is the sixteen semiconductor modules in the previous exhibit, because interpretable structure recovered without labels is hard to produce by accident.

Reference

Commands, the knobs that matter, where each equation lives, and the one install that silently changes what the system can discover.

EXHIBIT • RUN COMMANDS

COMMAND	WHAT IT DOES
<code>python train.py --config configs/small_scale.yaml</code>	Vanilla JEPA on CIFAR-10
<code>python train.py --config configs/small_scale.yaml -tcd</code>	Same, with the recursive loop live
<code>python -m experiments.two_rooms.run --config configs/two_rooms.yaml</code>	Gridworld, vanilla vs TCD side by side
<code>python -m experiments.image.run --config configs/small_scale.yaml</code>	CIFAR-10 comparison experiment
<code>python train_manifold.py --config configs/manifold.yaml --tcd --eval</code>	Graph mode plus the full evaluation suite
<code>python train_distributed.py</code>	DDP variant; <code>scripts/slurm_multinode.sh</code> for clusters
<code>python scripts/gnn_baselines.py</code>	GAT / GCN / GraphSAGE on the same splits
<code>python scripts/analyze_modules.py</code>	Inspect and name crystallized modules

Any config key is overridable inline: `python train.py --config configs/small_scale.yaml training.epochs=10 tcd.max_modules=16`. Every config passes through `validate_config()` before a model is built, and the run aborts on any error rather than training something misconfigured for an hour.

EXHIBIT • THE KNOBS THAT MATTER, AND WHICH PART EXPLAINS THEM

KEY	DEFAULT	EFFECT IF RAISED	SEE
<code>masking.pred_mask_scale</code>	<code>[0.2, 0.4]</code>	Harder task; more of the image hidden	III
<code>masking.num_enc_masks</code>	4	Effective batch multiplies; more views per target	III
<code>model.ema.start</code>	0.996	Slower target drift; more stable, slower to learn	II
<code>tcd.explore_every</code>	2–5	Fewer, cheaper exploration cycles	VIII
<code>tcd.crystallize_every</code>	5–10	Slower module growth	VIII

KEY	DEFAULT	EFFECT IF RAISED	SEE
tcd.langevin_steps	20–50	Longer trajectories; richer cloud, higher cost	V
tcd.langevin_step_size	0.01	Coarser walk; risks stepping past structure	V
tcd.persistence_threshold	0.2–0.3	Fewer modules — the bar rises	VI
tcd.max_modules	8–16	Larger population before pruning bites	VI
training.grad_clip_norm	1.0	Looser clipping; faster, less stable	IX

THE SINGLE MOST CONSEQUENTIAL INSTALL

Without `giotto-tda` or `ripser`, the crystallizer falls back to `scipy` single-linkage clustering. It logs one warning and keeps running — but two thirds of the module vocabulary become unreachable, and you will only ever see `mod_attractor_*` ids.

If a run produces no cycle or boundary modules, check the backend before changing any threshold. This is the failure mode most likely to be misdiagnosed as a tuning problem.

EXHIBIT • WHERE EACH EQUATION LIVES

EQUATION	IMPLEMENTATION
$E = \ p(s_\theta(x)) - sg[s_\xi(y)]\ ^2$	<code>core/energy_landscape.py:30</code> · <code>core/system1_encoder.py:84</code>
$\xi \leftarrow m \cdot \xi + (1-m) \cdot \theta$	<code>models/target_encoder.py:47</code>
$H_{dd} \approx [E(z+\epsilon d) - 2E(z) + E(z-\epsilon d)] / \epsilon^2$	<code>exploration/blank_space_detector.py:68</code>
$z_{\{t+1\}} = z_t - \eta \nabla E + \sqrt{(2\eta/\beta)} \cdot \epsilon$	<code>exploration/langevin.py:121–124</code>
$\beta(z) = \beta_0 / (1 + \text{blank_score})$	<code>exploration/langevin.py:151</code>
$F = (1/\sigma^2) \cdot J^T J$	<code>exploration/fisher_metric.py:87</code>
persistence = death – birth	<code>topology/persistence_diagrams.py:41</code>
dim → module type	<code>topology/feature_extraction.py:73</code>
$C(t) = \Delta M + KL(RIR') + \Delta S$	<code>core/recursive_loop.py:69–82</code>
out = base + $\alpha \cdot \text{gate} \cdot \text{LN}(\sum w_m \cdot m_m)$	<code>modules/dynamic_predictor.py:152–161</code>

EXHIBIT · TEST COVERAGE, 186 TESTS

SUITE	COVERS	TESTS
test_exploration.py	Exploration internals in depth	24
test_training_guards.py	NaN/Inf handling, spike detection	23
test_system3.py	PH backends, diagrams, factory, registry	18
test_modules.py	Module classes, router, dynamic predictor	17
test_system2.py	Blank space, Langevin, Fisher, explorer	16
test_integration.py	Full model, losses, schedulers	16
test_error_handling.py	Degenerate inputs and failure paths	14
test_system1.py	Encoder, context/target wrappers, hooks	11
test_property_based.py	Invariants across generated inputs	11
test_config_validation.py · test_recursive_loop.py	Config schema; loop orchestration	17
test_convergence.py · test_evaluation.py · test_checkpointing.py · test_distributed.py	Remaining subsystems	19

TCD-JEPA / Crystara — Apache 2.0. Builds on the JEPA paradigm from Yann LeCun and Meta FAIR: I-JEPA (Assran et al., 2023) for the core architecture, plus V-JEPA, V-JEPA 2 and eb_jepa.

Every code listing in this document is read verbatim from the repository at build time; the line numbers are the files' own.